Table of Contents

1 – Introduction 3
2 – System Architecture 4
2.1 – Overview of MVC and N-Tier Architecture 4
2.2 – System Design 4
2.3 – Technologies and Tools 9
2.4 – Design Choices 9
3 – Architectural Quality Attributes 10
3.1 – Availability 10
3.2 – Modifiability 11
3.3 – Testability 12
3.4 – Usability 13
3.5 – Security 14
3.6 – Summary 17
4 – Reflection 17
4.1 – Personal Experience and Role in the Project 17
4.2 – Problems I had and how I handled them 17
4.3 – Lessons Learned 18
4.4 – Benefits of Using MVC and N-Tier Architecture 18
4.5 – Overall Reflection 18
4.6 – Future Improvements 18
5 – Individual Contribution 19
5.1 – Overview 19
5.2 – Contribution Table 19
5.3 – Individual Contribution and Leadership 20
5.4 – Team Collaboration and Work Ethic 20
5.5 – Summary of Contribution and Effort 20
6 – Conclusion 20
7 – References 21
8 – Appendix 22
Appendix 1 – Home Page Wireframe 22
Appendix 2 – Nintendo Main Page Wireframe 23
Appendix 3 – Nintendo Consoles Page Wireframe 24
Appendix 4 – Nintendo Accessories Page Wireframe 25
Appendix 5 – Nintendo Games Page Wireframe 26
Appendix 6 – Account Login Page Wireframe 27
Appendix 7 – Account Registration Page Wireframe 28
Appendix 8 – Customer Account Page Wireframe 29

Appendix 9 – Admin Dashboard Page Wireframe 30
Appendix 10 – Basket Page Wireframe 31
Appendix 11 – Checkout Page Wireframe 32
Appendix 12 – Final System 33
Appendix 12.1 – Main Home Page 33
Appendix 12.2 – Nintendo Main Home Page 34
Appendix 12.3 – Xbox Consoles Page 34
Appendix 12.4 – PlayStation Accessories Page 35
Appendix 12.5 – Computer Games Page 35
Appendix 12.6 – Basket Page 36
Appendix 12.7 – Customer Dashboard 36
Appendix 12.8 – Admin Dashboard 37
Appendix 12.9 – Admin Customer Order Management 37
Appendix 12.10 – Admin Customer Order Management View Details 38

List of Figures

Figure 1 - High-Level System Architecture Diagram 4
Figure 2 - Entity Relationship Diagram (ERD) 6
Figure 3 - Class Diagram of Core Entities 8
Figure 4 - Error View Model 10
Figure 5 - AccountsController - Registration Logic 11
Figure 6 - AccountController - Database Context ApplicationDbContext 12
Figure 7 - User.cs - User Model 13
Figure 8 - Search Input Column 13
Figure 9 - Checkout Form 14
Figure 10 - AccountController - Password Hashing 14
Figure 11 - AdminController - Role Based Access 15
Figure 12 - Program.cs – Secure Session Configuration 16
Figure 13 - Register.cshtml – Password Strength Meter and Rules 16
Figure 14 - Home Page Wireframe 22
Figure 15 - Nintendo Main Page Wireframe 23
Figure 16 - Nintendo Consoles Page Wireframe 24
Figure 17 - Nintendo Accessories Page Wireframe 25
Figure 18 - Nintendo Games Page Wireframe 26
Figure 19 - Account Login Page Wireframe 27
Figure 20 - Account Registration Page Wireframe 28
Figure 21 - Customer Account Page Wireframe 29
Figure 22 - Admin Dashboard Wireframe 30
Figure 23 - Basket Page Wireframe 31
Figure 24 - Checkout Page Wireframe 32
Figure 25 - Final System – Main Home Page 33
Figure 26 - Final System – Nintendo Main Home Page 34
Figure 27 - Final System – Xbox Consoles Page 34
Figure 28 - Final System – PlayStation Accessories Page 35
Figure 29 - Final System – Computer Games Page 35

Figure 30 - Final System – Basket Page 36
Figure 31 - Final System – Customer Dashboard 36
Figure 32 - Final System – Admin Dashboard 37
Figure 33 - Final System – Admin Customer Order Management 37
Figure 34 - Final System – Admin Customer Order Management View Details 38

List of Tables

Table 1 - Contribution Table 19

1 – Introduction

The purpose of this report is to present a clear, detailed explanation of the GameHaven Online Shop, which is an online retail application that was constructed as a community effort. The GameHaven online shopping application appears and works like a legitimate online gaming store, featuring stuff like gaming toys, tools, and games. It also shows off some amazing program design principles like the MVC technique.

ASP.NET Core MVC is used to construct GameHaven, which keeps the appearance layer, business layer, and data layer independent from one other. The ORM within the application is Entity Framework Core, which enables for secure and effective management of a SQL Server database holding data about users, products, sales, carts, and administration. Bootstrap and CSS are used to construct the front-end user experience.

The system has possibilities for regular e-commerce procedures like browsing, choosing items, reviewing product details, storing a shopping cart, and even completing a transaction. An Admin Dashboard has also been combined to control items entirely using CRUD methods, same as in a genuine business and it also has a payment method.

There are four important aspects to this report:

- In section 2, I will present an overview of the system design, which contains pictures illustrating how MVC and n-tier structures were incorporated.
- In section 3, I will examine fundamental design quality aspects such as security, usability, modifiability, availability, as well as testability, coupled with the accompanying tools.
- In section 4, I will discuss my ideas regarding the development process, some issues/problems that were experienced, as well as some things that were obtained by implementing the MVC & layered approach.
- In section 5, I will provide a comparison of team member's obligations to those of the team leader is described.

Finally, it has been a beneficial experience developing the GameHaven Online Shop. It has taught us a lot about program construction, cooperation, as well as debugging. It has also enabled us to use our skills towards developing a functional system, controlling it, as well as following software development practices.

2 – System Architecture

2.1 – Overview of MVC and N-Tier Architecture

The Architectural design of GameHaven Online Shop are built upon two related design patterns that are ASP.NET MVC and N-Tier. These two design patterns are utilised to create a clean system utilising which we are able to develop an independent system.

There are three pieces to an application employing an MVC framework.

- Models link to facts and business processes. Classes like Product, User, Order, BasketItem are simply mapped to SQL Server using Entity Framework Core.
- Views are used to present facts to individuals. Razor-based .CSHTML pages are used for altering content relating to product browsing, operating the basket function, and processing checkout.
- Controllers control information flows. Controllers like HomeController, ProductController, or AccountController receive user behaviours or inputs, verify them, and depending on those behaviours or inputs, either conduct View logic or Model logic.

In addition to employing MVC, another element of the system design is the usage of N-Tier architecture which breaks the application into:

- A Presentation Layer which is for Razor, HTML, CSS, Bootstrap
- A Business Logic Layer which is for controllers, validation logic, decision-making logic, view models)
- Data Access Layer which is for connection between Entity Framework Core and SQL Server)

The advantage of this strategy is that modifications made to other layers, such updating the user interface or modifying database structures, cannot meddle with other levels. As a consequence, maintenance or altering additional layers in the future will become straightforward.

2.2 – System Design

The system architecture itself displays a clear data flow in terms of the MVC structure.

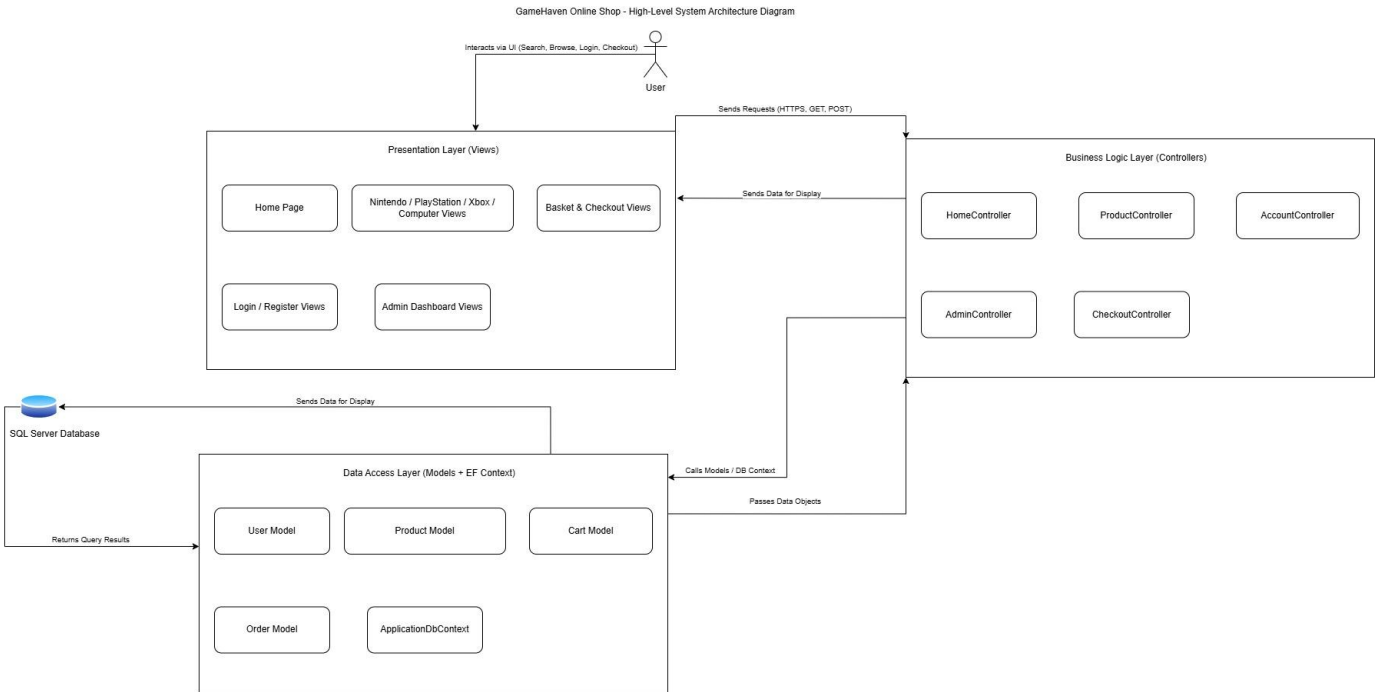


Figure 1 - High-Level System Architecture Diagram

Figure 1 is demonstrating a High-Level System Architecture Diagram for our E-Commerce website named GameHaven online store. The image above depicts how data transfers suitably depending on this on this diagram. The diagram will initially start with the user. The user will have the power and ability to conduct the following interactions once they're clearly on the GameHaven website, search to locate which games, accessories and consoles they desire, browse to check items pricing, log in to monitor and see personal account information and prior account history. And lastly checkout to input personal information and card details to buy the products you want.

The Presentation Layer is responsible for handling all of these key interactions and will also show you the views the user will be first able to see such as the Home page, each category page for its own products(Xbox, PlayStation, Nintendo and computer), Basket and checkout pages, admin dashboard page and login and registration page. This layer will reveal what the user will be able to view right at the beginning of the screen and be able to click on and any pages that the user chooses.

The Business Logic Layer will then follow up on whatever inputs the user has provided on the website. Once the user has made an input, the HTTP will then receive a request to the business logic layer, which has 5 system controllers. Admin controller, home controller, product controller, account controller and checkout controller. The fundamental function for all of these controllers is to be able handle any requests made by the user and apply them with business rules such managing order processing, product filtering, cart management, and authentication to then compute the next procedure necessary to finish these tasks.

The main 5 controllers which was indicated for the business logic layer also will then talk to what is called the Data Access Layer if the system requires any important information from the database. The data Access Layer or DAL simply is able to allocate simple access to data that is kept within a storage. This layer will include the following Entity Framework models such as the product, order models cart, and user combined with the ApplicationDbContext. These capabilities will be able to manage and deal with how the data is utilised and illustrate the program interacts with the SQL server database.

The ApplicationDbContext simply translates the requests from the controller into SQL queries, Updates or restore the data necessary and then puts it back into a structured data object in the controllers. The controllers then get to utilise this data to generate a proper view model, which they then continue to provide to the presentation layer to show the related results to the user.

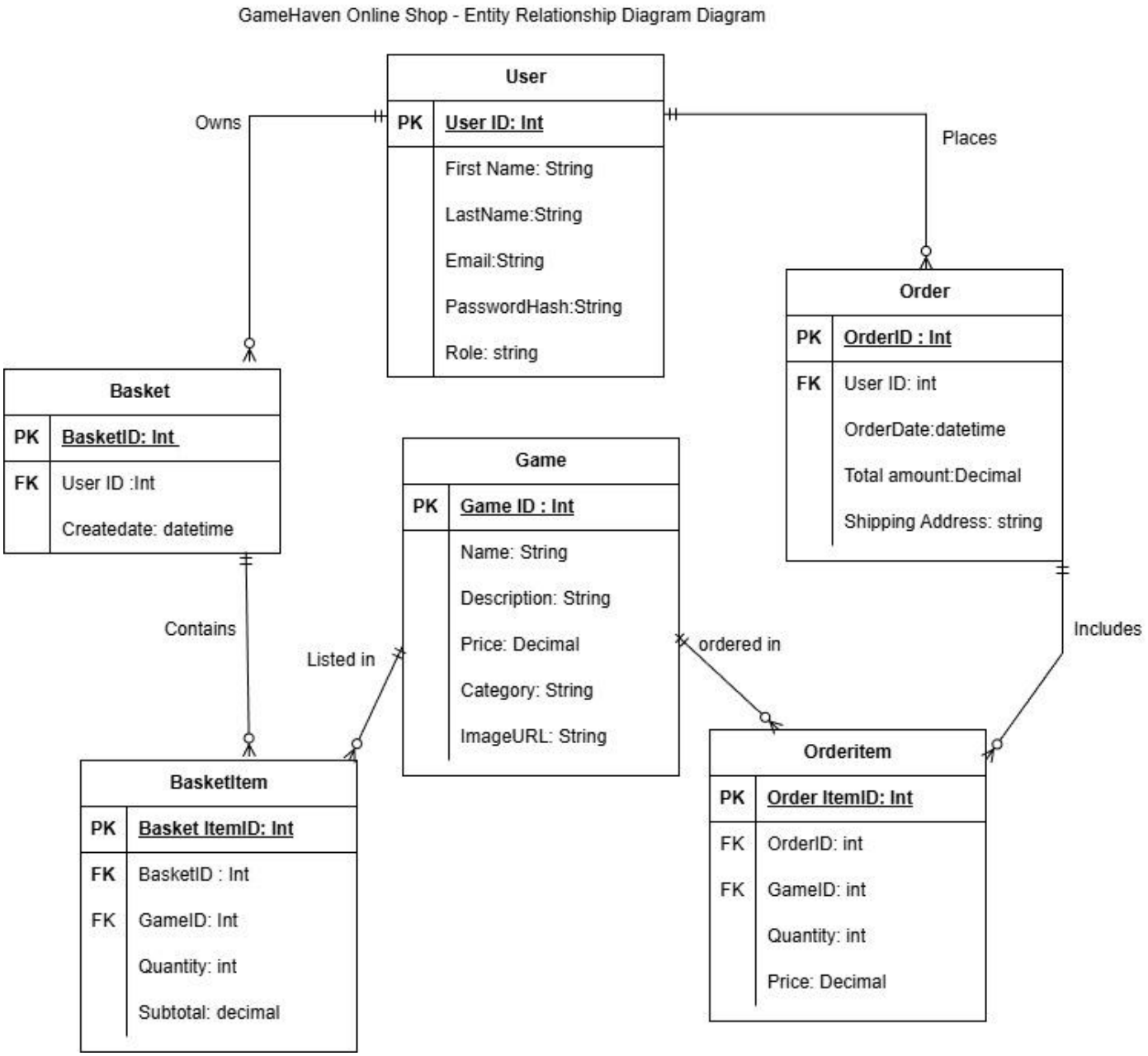


Figure 2 - Entity Relationship Diagram (ERD)

In figure 2, which is an Entity Relationship diagram for GameHaven, it is evident how all of the databases link to one another. It contains 6 entities (User, Basket, Basket Item, Game, Order and Order item). Each entity will have its own Primary key and Foreign key if required. The connections to all entities are 1 to 1 and 1 to many relationships to illustrate how the actual program operates in real time. 1:1 interaction helps making ensuring every user has a basket. On the other side, 1 to many relationships assist the user to be able to have multiple orders and baskets that include many goods.

The User object holds the data of a client such as their first name, last name, Email,Passwordhash and role. Each user will have its own User ID which will serve as the main key. The user will possess a shopping basket, which basically means the user will have 1 basket associated to their account where the items will be kept. Users will also be allowed to make as many orders as they want to, once they've processed the orders that will display that it's connected to not the UserID. This will illustrate how each entity may be connected to each other by performing purchases.

The Basket Entity functions as the consumers shopping cart. This comprises BasketID, UserID of the individual, and the date that it was made. In the Basket entity the UserID contains the foreign key because its tied to the user demonstrating that its associated or linked to each other. The BasketID is the main key to demonstrate that it is its own unique key which enables the database itself to add, remove and store baskets appropriately.

Basket Item entity has BasketitemID with its own unique primary key. Game ID and Basket ID are both Foreign Keys that are related to the Basket and Game entities. Alongside each are the quantity and the subtotal. The includes connection implies that the system will recognise what item/product the user is going to purchase dependent on the quantity.

The Game Entity contains within its own entity, the Game ID, which is its own main unique key. Then it also contains the name of the game, Description of the game, price of the game, Category of the game and lastly the picture of the game. The games may be in several baskets because of the mentioned in connection with the GameID as the foreign key in the BasketItem object. Games may also be ordered in numerous items using the orderitem entity which will also be able to hold the GameID which contains the foreign key. Both of these interactions illustrate how 1 item may be in several baskets and with many orders completed orders in the system.

The order entity contains the OrderID with it being a primary key, The UserID which is a foreign key that's connected back to the User entity, date of the order, total cost and delivery Address. This reveals the information of the processed order so that the user may know that the order has been properly completed. OrderItem has the main key itself that demonstrates it has its own unique key. Order ID is regarded a foreign key because of its related connection to order, same thing applies to GameID since it ties to the full game entity. The includes relationship gives a clear and straightforward history of orders that the user has placed and it contains the receipts to provide clear proof.

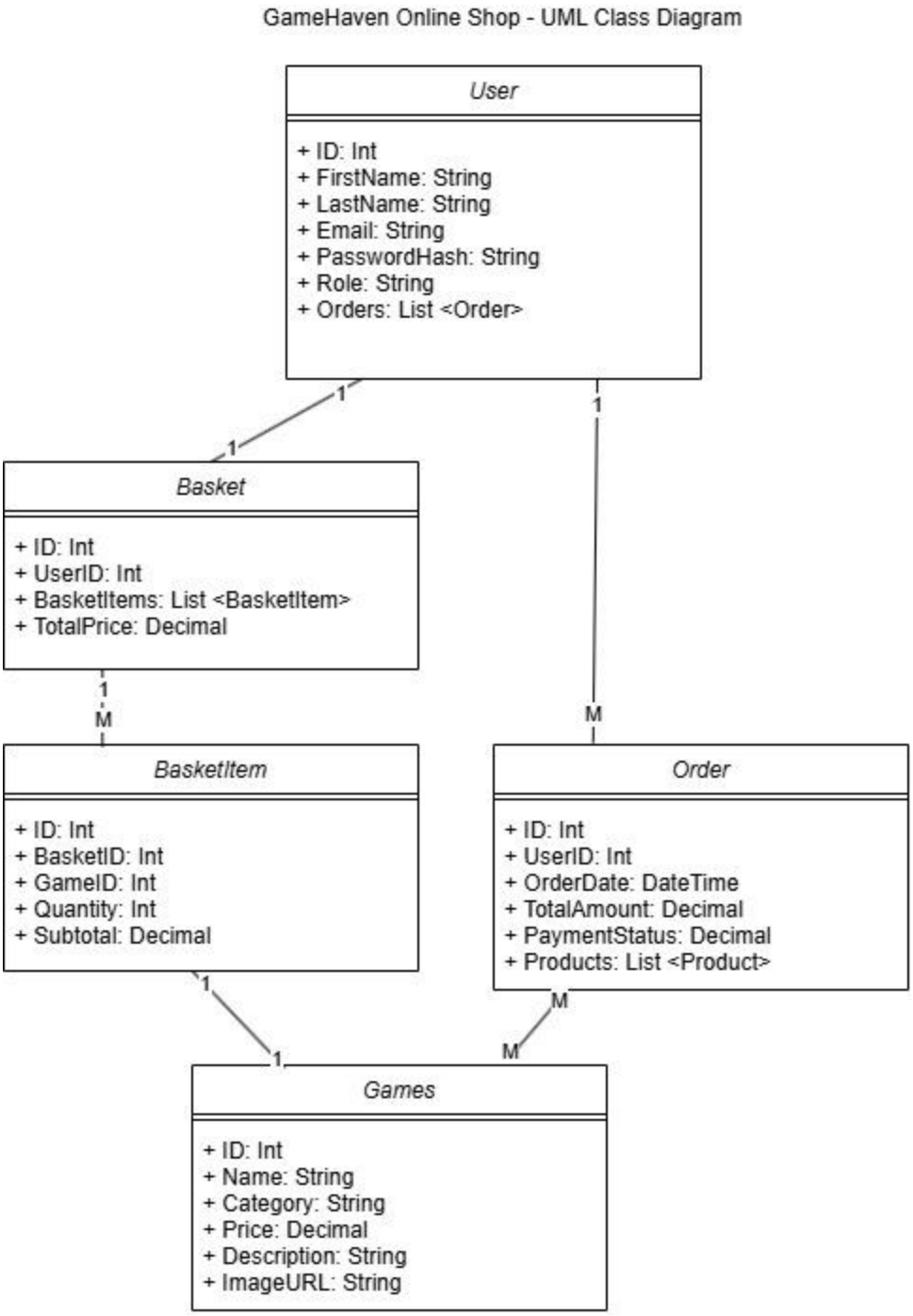


Figure 3 - Class Diagram of Core Entities

The User Class displays each consumer who creates the account on the gaming haven website. This class is basically responsible for keeping data such as firstname, last name, email, passwordhash, orders and role that may be allocated as the user or the admin. A user is allowed to generate many orders from the 1 to many relationship which implies that the user will have one basket which may include several products that they would want to add before they make their final purchase.

Basket Class represents the consumers shopping cart. The basket belongs to the customer account which is related to the user class. The basket will have various things that may display or reflect the quantity of games the user is adding to its cart. The basket class contains ID, User ID which is related to the user,(Foreign key) Basketitem and total price. The one to one link between the user and basket signifies that basket belongs to one user.

BasketItem Class indicates the item the user has picked for their basket. The Basketitem Class comprises ID, BasketID, GameID, quantity and subtotal. Each item signifies denotes the amount of games or goods added

This is just because the basketitem might be modified to other goods. So, because the basket includes diverse things. In this situation, the connection between the basket and basketitem would be one to many.

Order Class can maintain and save information about any purchases the user has made after they have made the payment. The order class contains the ID, User ID which again refers to the user, Orderdate, Total amount, payment status and goods The order class will monitor the time and date the order was made, how much was paid and the status to decide wether the payment was successful or not. Order class have several things, shown as a list of items. What this basically implies is that the user is allowed to buy several games in one transaction which signals from the user to order it's a one-to-many connection and order to games is many to many.

Finally, the Games Class represents things that are available on stock Each game will have its own distinct name and it will also be related to 4 different categories such as Xbox, PlayStation PC and Nintendo. Games class will have ID, name, category, price, description and ImageURL that will be clearly presented to the user on accessing the gaming website GameHaven. The link ship between games class and basket items is 1 to many since 1 solitary game might be exhibited in numerous baskets. In plain words, numerous people may add the exact same game to their own baskets as many times as they like. Games also relates to order which implies it's a many to many connection since the user may order once with many games as they wish.

These design decisions emphasis a rigorous segregation of concerns. The controllers are responsible for input processing and decision logic but never interact directly with database searches. Entity Framework Core takes care of searches and transaction management to prevent SQL attacks and maintain everything aligned with best practices in software development.

2.3 – Technologies and Tools

These are the list of technologies and tools used for building of GameHaven Website:

- ASP.NET MVC Framework: Offers quite a number of functionality associated to developing ASP.NET MVC projects.
- Entity Framework Core (ORM): Enables secure and quick database access in SQL Server using object-oriented programming approaches instead of using SQL.
- SQL Server: contains all permanent data like users, products, sales, and basket items.
- Bootstrap: Provides a flexible and available user experience for PCs and mobile devices.
- Microsoft VS 2022: Used for writing, testing, and debugging.
- GitHub: The project employed GitHub for retaining its version control. GitHub provided communication amongst team.

These development tools were adopted to facilitate speedy development, safe coding practices, and a common graphical user interface.

2.4 – Design Choices

We opted to construct using ASP.Net MVC as it is compatible with C# programming. Its architecture is versatile and allows us to develop an application in a logical approach. Its design pattern is adaptive, which helps us to develop an application without any repeated work on a team when working on multiple components. ASP.Net MVC is testable. Its design pattern makes it applicable for any application associated to product ideas or user evaluations.

Entity Framework Core was selected to doing SQL due of these reasons:

- Automatic query generation
- Good protection against SQL injection attacks
- Easy changes to change schemas
- A clean object-oriented interface

Moreover, adopting Bootstrap for designing the frontend meant that there was uniformity in terms of style across all the sites. Additionally, there was no need to set up CSS to make sites customisable.

At all levels, design decisions centred on availability, modifiability, testability, usability, security and future-proof expansion. In doing so, not only does GameHaven fulfil its intended use case well as a comprehensive e-commerce system prototype, but it's poised to handle difficult features like a true payment system or role-based displays.

3 – Architectural Quality Attributes

This section shows how the design choices taken in delivering the GameHaven Online Shop helped to address availability, modifiability, testability, user-friendliness, and security. These needs are addressed not only because we employed a solution based on both an MVC model and N-Tier architecture but also because we actively developed our project to concentrate on these requirements.

3.1 – Availability

Availability was a vital component to ensuring that consumers are not interrupted while perusing items, altering their carts, or making payments. ASP.NET Core MVC helps to accomplish these aims utilising its request flow feature. The system provides an Error ViewModel coupled with various error pages listed below:

```
namespace GameHaven_Online_Shop.Models
{
    // View model used for displaying error information to the user
    3 references
    public class ErrorViewModel
    {
        // Unique identifier for the current request (used for tracking/debugging)
        3 references
        public string? RequestId { get; set; }

        // Returns true if a RequestId exists - helps show/hide it in the error view
        1 reference
        public bool ShowRequestId => !string.IsNullOrEmpty(RequestId);
    }
}
```

Figure 4 - Error View Model

The principal tactic adopted in this scenario is Exception Handling and Fault Isolation. The approach employs an Error ViewModel to record exceptions, therefore avoiding system failure. The method was accepted for its utility in giving a gracefully degraded system, allowing unplanned faults and enabling a smooth transition to a user-friendly error page instead of a normal exception screen. The other possibilities for availability were abandoned as they demanded considerable price, which is unacceptable for a small-scale e-commerce prototype corporation.

Every error in this model receives a unique request ID. This makes tracking errors easier. The system does not shut down if there are unanticipated issues but instead takes visitors to an error-friendly site.

N-Tier design for availability occurs due to splitting. Any faults pertaining to the Presentation Layer will not create downtimes in data access functions since database functionalities are not restricted owing to retries introduced to Entity Framework Core. As a result, the user functions are solid.

The tactic employed in this design is Isolation by Layered Architecture Isolation. To achieve this, the layers include the Presentation, Business, and Data Access Layers. In this method, if a layer fails, the other layers will not be harmed, hence boosting availability. This method was adopted because of its simplicity. For availability, Microservices or Message Queuing may have been employed, but they are challenging, consequently their rejection as a suitable approach for a prototype MVC solution deployment.

3.2 – Modifiability

One method modifiability is made feasible is because of the adoption of the MVC design pattern. As its name says, Models, Views, Controllers are separated into distinct components. In this approach, functionality may be introduced without impacting other components that are not related to those functions.

One prominent example can be found in the AccountsController's Registration function. It's attended to utilising a special async function:

```
// Register

// POST: Handle registration submission
[HttpPost]
[ValidateAntiForgeryToken]
0 references
public async Task<IActionResult> Register(User user, string password)
{
    if (!ModelState.IsValid)
        return View(user);

    // Exclude seeded admin
    bool isSeededAdmin = user.Email.Equals("admin@gamehaven.com", StringComparison.OrdinalIgnoreCase);

    // Check if user already exists
    bool exists = await _context.Users.AnyAsync(u => u.Email == user.Email);
    if (exists)
    {
        ModelState.AddModelError(nameof(user.Email), "Email already registered.");
        return View(user);
    }

    // Apply password validation only for non-admin and new users
    if (!isSeededAdmin)
    {
        var passwordErrors = ValidatePassword(password);
        if (passwordErrors.Any())
        {
            foreach (var error in passwordErrors)
                ModelState.AddModelError(nameof(password), error);
            return View(user);
        }
    }

    // Assign role based on email domain
    user.Role = user.Email.EndsWith("@gamehaven.com", StringComparison.OrdinalIgnoreCase)
        ? UserRole.Admin
        : UserRole.Customer;

    // Hash the password
    user.PasswordHash = _hasher.HashPassword(user, password);

    // Add user to database and save changes
    _context.Users.Add(user);
    await _context.SaveChangesAsync();

    // Store session values
    HttpContext.Session.SetString("UserEmail", user.Email);
    HttpContext.Session.SetString("UserRole", user.Role.ToString());
    HttpContext.Session.SetString("UserFirstName", user.FirstName ?? "User");

    // Redirect based on role
    return user.Role == UserRole.Admin
        ? RedirectToAction("Index", "Admin")
        : RedirectToAction("Index", "Customer");
}
```

Figure 5 - AccountsController - Registration Logic

The tactic that has been selected is Separation of Concerns and Encapsulation of Change. The technique requires separating the challenges associated to registration into an asynchronous activity, and this will make any feature additions, such multi-factor verification, easily without rewriting much code. The approach was adopted thanks of decreased coupling and improved flexibility. Other methods, including Service Oriented Architecture, are ruled out as they are resource-intensive given the limited context at play at the level of this model-view-controller application.

In fact, this design style makes it simpler to add upgrades in the future like MFA or role-based registration since these changes are offered by expanding previously existing patterns. Asynchronous programming or async/wait features enhances both modifiability and scaling since handling concurrent calls becomes efficient.

Entity Framework Core introduces modifiability features utilising abstraction. Any update to the database to meet characteristics such as DateOfBirth or ProfilePicture necessitates adjustment to the model. Finally, updates to databases are accomplished using Entity Framework transformations.

Here, the tactic is Data Abstraction and Mapping through the ORM layer (Entity Framework Core). It removes the complexities of the database from the business layer, as well as offers flexibility by making use of migration scripts. The strategy was short-listed basing on its promise to maintain and upgrade the schema periodically. Other strategies, like Direct SQL Scripting, are invalidated owing to their implications and destructive impacts related to future modifications to the data layer.

Dependency Injection (DI) is another key component. With DI's ability to include ApplicationDbContext and PasswordHasherUser into its components, interaction between components becomes minimised to a considerable amount. Also, future adjustments might be performed without much labour. For example, adopting Azure security.

The tactic that used is Dependency Injection or DI. The approach provides loose coupling and component substitution at Runtime. The approach Dependency Injection was adopted as it supports modularity and facilitates modifications later. Other strategies, such as using a Service Locator Pattern, aren't allowed because they tend to obscure dependencies, which makes testing and maintenance difficult.

Combining all of them leads to an efficient system which can be operated effectively and even personalized in the future, if desired.

3.3 – Testability

Testability elements are integrated into the project utilising architectural design. In the MVC design pattern used in coding, there are advantages of separate components like models, controllers, or views being tested.

Dependency Injection, this feature makes testing easier. Take AccountController for example. It accepts ApplicationDbContext as a prerequisite in its constructor:

```
// Controller for handling user account actions like login, register, and logout
1 reference
public class AccountController : Controller
{
    // Database context for accessing users
    private readonly ApplicationDbContext _context;

    // Password hasher for hashing and verifying passwords
    private readonly PasswordHasher<User> _hasher = new();

    // Constructor with dependency injection for the database context
    0 references
    public AccountController(ApplicationDbContext context)
    {
        _context = context;
    }
```

Figure 6 - AccountController - Database Context ApplicationDbContext

The tactic adopted in this case is Interface and Dependency Abstraction. The strategy involves introducing dependencies, such as that of the database context, which could be replaced by an interface that is either a mock object or an interface for an in-memory database, as a testable strategy, a choice informed by how applicable this is when using principles of Test-Driven Development, TDD for short. The techniques Runtime Instrumentation may have been suitable but offer needless complexity.

This enables feasible testing the controller using an in-memory database system instead of utilising a real SQL Server database. The functionality of creating a user, logging in, or logging out will therefore be evaluated.

Also, the product controller employs Dependency Injection. Therefore, its CRUD operations are readily evaluated using a false or in-memory data source.

Once again, Dependency Injection for Test Substitution is applied. It provides the replacement of the real repository with a test replica, hence making practicable the unit testing independent of other components. The approach was utilised because to the ease associated in regression testing. Other options, like External Service Virtualisation, are not appropriate, given the lack of API requirements involved.

Testability is further increased by data comments supplied to models. For example, in the User model:

```
    // Model representing a user
    9 references
    public class User
    {
        // Unique identifier for the user
        4 references
        public int Id { get; set; }

        // User's profile information
        11 references
        public string FirstName { get; set; } = "";
        8 references
        public string LastName { get; set; } = "";

        // User authentication information
        15 references
        public string Email { get; set; } = "";
        5 references
        public string PasswordHash { get; set; } = "";

        // User's home address
        7 references
        public string? HomeAddress { get; set; }

        // Role of the user (Customer or Admin)
        7 references
        public UserRole Role { get; set; } = UserRole.Customer;
    }
}
```

Figure 7 - User.cs - User Model

The tactic, in this case, is Built-in Validation and Assertion Checks. Annotations make verification a build and runtime process, making automation easy. The solution was used because it Make an interface between ASP.NET Core, leveraging the Validation Features. Other options, Custom Validation Scripts, are avoided as they are redundant and provide room for a Human Factor.

These properties enable testing validation rules utilising automation testing without conducting out human validation. ASP.NET Core gives opportunity for verifying additional aspects like model binding validity and model state issues. These provide appropriate functioning for action inputs utilising forms in case there are no values or improper values submitted.

3.4 – Usability

Making the site functional was a crucial objective to enable users a seamless buying experience. Every site was developed utilising Web Usability principles.

One of the key adjustments made in terms of usefulness is connected to the search option for product information for instances in the Nintendo, Xbox, Computer and PlayStation pages. The search term input is supplied to controllers via query strings:

```
<!-- Search Input Column -->
<div class="col-md-8">
    <input type="text"
        name="q"
        value="@(ViewData["CurrentSearch"] ?? "")"
        class="form-control"
        placeholder="Search" />
</div>
```

Figure 8 - Search Input Column

The most significant strategy being utilised is Input Simplification and Responsive Feedback. The search box is directly coupled to controller logic via query strings, providing rapid dynamic input to users. The strategy was picked thanks of the gain in learnability and responsiveness that comes with its execution. Other solutions, such as Server-side Filtering Only, are avoided since they degrade responsiveness and delay input to consumers.

This makes continual searching feasible. The search term will also persist after a page restart using ViewData["CurrentSearch"]. Bootstrap offers you adaptability to varied sizes. It provides mobile-friendliness.

Usability is also addressed in the checkout function. In reality, the CheckoutViewModel employs checking features to support users in entering in payment and shipping information:

```
// Represents checkout form data and includes validation logic
4 references
public class CheckoutViewModel : IValidatableObject
{
    // Email field (only shown when user is not logged in)
    [EmailAddress, Display(Name = "Email")]
    9 references
    public string? Email { get; set; }

    // Credit card number (required)
    [Required, Display(Name = "Card number")]
    5 references
    public string CardNumber { get; set; } = "";

    // Card expiry date in MM/YY format (required)
    [Required, Display(Name = "Expiry (MM/YY)")]
    9 references
    public string Expiry { get; set; } = "";

    // CVV security code (required)
    [Required, Display(Name = "CVV")]
    5 references
    public string CVV { get; set; } = "";

    // Used to determine if email input is needed
    6 references
    public bool IsLoggedIn { get; set; }
```

Figure 9 - Checkout Form

The strategy that will be adopted is Form Validation and Immediate Error Feedback. The strategy will utilise DataAnnotations, which are found in ASP.NET, including annotations like [Required] or [EmailAddress]. The strategy will allow users instant feedback if they have entered any incorrect data. The strategy will allow users instant feedback if they have entered any incorrect data. The strategy will be adopted as it will save user's time, as opposed to other strategies, like Server-side Error Display After Submission, which will interrupt user's flow.

The usage of [Required], [EmailAddress], and display attributes makes problems simple to spot. The IsLoggedIn functionality makes user data autofill simple as users are verified before data autofill.

These designs enable both idle users and serious customers to utilise the system effortlessly and effectively.

3.5 – Security

Multi-level security guard's user details, identity, and system safety. Passwords matter most for user registration. People never keep them plaintext. ASP.NET Core PasswordHasherUser employs one-way hashing:

```
// Password hasher for hashing and verifying passwords
private readonly PasswordHasher<User> _hasher = new();
```

Figure 10 - AccountController - Password Hashing

The tactic employed in this circumstance is One-Way Encryption (Password Hashing). One-Way Encryption was adopted as a solution as it makes reversibility difficult and guarantees that a user's data is safeguarded even if a breach occurred. Other approaches, particularly Symmetric Encryption, are ruled out as they require a key for decryption, making decryption conceivable.

In reality, employing hashed passwords implies that regardless of whether or not a database is accessed, password reversals are impossible. Such a technology makes dealing with GDPR easier.

To further secure boss duties, there's Role-Based Access Control (RBAC). In AdminController, entrance is denied according to user privileges:

```
// Constructor with dependency injection for the database context
0 references
public AdminController(ApplicationDbContext db)
{
    _db = db;
}

// Check if the current user is the admin
9 references
private bool IsAdmin()
{
    var email = HttpContext.Session.GetString("UserEmail");
    return email != null && email.ToLower() == "admin@gamehaven.com";
}

// Admin dashboard - list all games
3 references
public IActionResult Index()
{
    if (!IsAdmin()) return RedirectToAction("Index", "Home");

    var games = _db.Games.ToList();
    return View(games);
}
```

Figure 11 - AdminController - Role Based Access

The security tactic used in this example features Authorisation using Role-Based Access Control (RBAC). The technique will guarantee that all accesses are done by approved admin users alone, a strategy that was adopted given its support for the principle of least privilege and the ease with which it can be kept updated by ASP.NET Core Identity. Other strategies, including Attribute-Based Access Control (ABAC), have been ruled out as they will make the authorisation approach unduly onerous, a strategy that is not fit even for a small-scale such as this one.

This makes sure that non-authorised persons cannot see admin sites, not even directly via URLs.

In order to increase security while processing, safe session handling is supported in the Program.cs file. The session cookies are set to HTTP-only and essential with a timeout set to 30 minutes to prevent session hijacking:

```
<!-- Password Strength Meter visual progress bar -->
<div class="mb-2">
    <div class="progress" style="height: 10px;">
        <!-- This inner bar's width updates dynamically via JavaScript -->
        <div id="passwordStrengthBar" class="progress-bar" style="width: 0%;"></div>
    </div>
</div>

<!-- Password rules checklist with visual feedback -->
<ul class="list-unstyled small mb-3">
    <li id="ruleLength" class="text-danger">❌ At least 8 characters</li>
    <li id="ruleUpper" class="text-danger">❌ At least one uppercase letter</li>
    <li id="ruleLower" class="text-danger">❌ At least one lowercase letter</li>
    <li id="ruleNumber" class="text-danger">❌ At least one number</li>
    <li id="ruleSpecial" class="text-danger">❌ At least one special character</li>
</ul>

<!-- Displays any validation messages returned by the model -->
<div asp-validation-summary="ModelOnly" class="text-danger text-center mb-3"></div>
```

Figure 12 - Program.cs – Secure Session Configuration

The tactic adopted is Session Management and Timeout Control. It assures that the duration of the session is constrained and that the cookies are HTTP-only. The strategy minimises the threat of session hijacking attempts. The approach was adopted thanks to its simplicity and inbuilt mechanism to secure a session. Other options, such as Token-Based Authentication (JWT), were ruled out due to their complexity, even if they are capable of supplying additional means to secure a session.

It assures that login credentials and user information are not disclosed.

Along with these security measures, anti-forgery validation tokens ([ValidateAntiForgeryToken] property) are supplied for private comments to minimise vulnerability to cross-site request forgery attacks (CSRF).

In terms of UI usability, password security increases with password strength meter checks. There's real-time approval for regulations in the Register View:

```
// Session support
builder.Services.AddDistributedMemoryCache(); // In-memory cache for session
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(30); // Session timeout 30 minutes
    options.Cookie.HttpOnly = true; // Cookie is HTTP only
    options.Cookie.IsEssential = true; // Mark cookie as essential
});
```

Figure 13 - Register.cshtml – Password Strength Meter and Rules

The tactic used, therefore, is User Input Hardening & Validation. The approach optimises both security and usability by being demanding on passwords at the point of entering. The approach was adopted because to its significance in improving user's awareness, as well as having negligible influences on the performance layer. Other methods, including Forced Password Rotation, are avoided because they continue to hinder usability, resulting in worse secure behaviour, including repeated passwords.

It provides users fast feedback to enable them to develop better and more secure passwords to prevent using weak passwords.

Dependency injection not only makes testing simpler but also enhances security as there are no hardcoded URLs or passwords. These services are developed in a safe way, and one may alter

These parts working together imply that there is a multi-level security framework in place which protects personal information, guarantees powerful access control features, counters known attack spots, and promotes prudent user behaviour for the whole system which makes up GameHaven.

3.6 – Summary

In terms of the five quality attributes described earlier availability, modifiability, testability, usability, and security it's reasonable to claim that in all of these areas, the online store in GameHaven does follow these design principles. It does so in terms of worry division, inputs, review procedures, mobile-friendliness, and multi-level security.

4 – Reflection

4.1 – Personal Experience and Role in the Project

Working on the GameHaven Online Shop project was incredibly useful for me. As a Lead Developer for this project, some important system components that came under my obligations were Admin Dashboard functionality, password validation/password strength meter, and entire database-related functionality employing Entity Framework Core. Apart from these obligations connected to server operations, bespoke views for the website had to be designed.

The Admin Dashboard is one that I found very pleasant to construct since it includes a problem in real life with a known knowledge of the levels associated in MVC. The notion of looking for data using SQL Server models, then updating it by utilising controllers, followed by a final procedure to show data via Views was quite creative. The completion of CRUD operations helps to highlight processing of data in MVC models.

One such significant project was developing on a password checker program with real-time password strength recommendations. The combination of JavaScript code with password check code built using C# aided me in understanding how I might maintain a balance between UX and password security.

Apart from these classes, I have found learning about migrations, connections, and restrictions in Entity Framework Core to be illuminating with respect to design and discovery of databases using LINQ. This has helped me recognise that I need a solid data access layer in order for everything to work.

4.2 – Problems I had and how I handled them

As part of attempting to resolve several flaws uncovered in the project, one was data accuracy between the staff and client sides. Tests indicated there was a lag involving asynchronous activities connected to data not being loaded rapidly. Improving this problem demands improving code linked to controllers.

The second concern was updates in the model that frequently disagreed with the Entity Framework move history. These commonly make blunders in modifying the information. These difficulties might be rectified by understanding how to clean or restart migrations, reseed the database, or make sure that there was a reliable procedure for migrations. These occurrences have showed me to respect versioning not just in programming but also in databases.

The design of Admin Dashboard was another use case which presented user experience design issues. Initially, my design was busy. Later, I enhanced its design using Bootstrap's grid system.

4.3 – Lessons Learned

From this project, I realised that planning and structure are equally as vital as programming. The adoption of the MVC model underlined how effective well-structured cooperation can be when there are no conflicts amongst team members.

The connection with EF Core provided to increased understanding about data integrity difficulties associated to input sanitisation security hazards against SQL assaults. Apart from these abilities, there was exposure to advantages using small-scale testing to verify whether there are difficulties before to continue. My expertise of GitHub version control was enhanced.

4.4 – Benefits of Using MVC and N-Tier Architecture

Implementing both MVC and N-Tier architectures functioned as a significant architectural aspect. The MVC technology adoption helps to split responsibilities across models, controllers, and views. We utilised MVC technology to divide responsibilities across models, controllers, and views. Changes in Admin Dashboard design needed updates solely on views.

The N-Tier technique is a supplement to MVC since it gives another option to divide up the application process with tiers that effect display, business logic, and data access. Segregation makes it easy to have greater leeway to develop since tiers may possibly produce a mobile version with minimal alteration.

These patterns increased security for input validation. They verified that permission and business validations for this application are done in a secure way.

4.5 – Overall Reflection

In conclusion, I can now declare with confidence with what I have done and learnt during this project. Throughout this assignment I now have a better grasp of how to program in C#. Furthermore I have also learnt how to manage difficulties that happen and I am now more confident with it, I also now realise that I need to pay attention to details and vital information and be able to adapt to any changing environment.

I was in a position to enhance my skills as a developer because this particular project not only gave me opportunities to work on complex features but even supervise a team.

4.6 – Future Improvements

If further work were possible, I would add research capabilities into the Admin Dashboard. These features would include sales images and stock number graphics.

Other improvements would be wish lists and product reviews. In future versions, I would rethink the system to work as an API-centric platform using ASP.NET Web API to allow interaction with mobile apps.

Also, adding multiple manager jobs with different levels of rights would help to making the system a full business product.

This project has affected my thoughts concerning web creation. Now, I am not only thinking like a computer programmer but like a person who has to create a full system.

5 – Individual Contribution

5.1 – Overview

The GameHaven Online Shop was completed team-based. Every team member was allocated a set of responsibilities according to their competence. As Lead Developer for the project, I was responsible for system design, code standards, as well as interface demands.

Below is a table describing major contributions of team members, areas of work, as well as an approximate estimation of their effort in terms of 1 to 5, where 5 indicates a complete commitment of effort.

5.2 – Contribution Table

Member Name	P Number	Main Contributions	Effort (1-5)
Shyam Dattani	P2785659	Lead Developer, Responsible for entire system architecture, including building out Admin Dashboard, password validation and strength meter, as well as database integration, and managed version control through GitHub while ensuring code uniformity.	5
Gavin Bisla	P2796362	Implemented registration, login, and Customer Profile page functionality, as well as checkout process logic and user session functionality related to payments.	5
Sadrach Gyimah	P2801486	Developed Xbox controller designs and opinions, implemented a search bar functionality across various pages of a product filter system.	5
Hakeem Blackson	P2724555	Developed controllers and views for the homepage and created controllers and views for PlayStation and Computer pages, also implemented Guest Checkout functionality across all platforms.	5

Table 1 - Contribution Table

5.3 – Individual Contribution and Leadership

As a Lead Developer, I have built the MVC framework for this project. I have constructed this project's database. The Admin Dashboard, password strength, and role-related functionality are key components built by me.

I was also a GitHub Lead. The position required branch maintenance, dealing with merge problems, code reviews, and code standard enforcement. Besides these, additional tasks that I had included correcting controllers, guaranteeing route validity, and checking components for user sessions.

The interaction enabled me in ensuring all functionality spanning from CRUD in the basic module down to checkout was merged.

5.4 – Team Collaboration and Work Ethic

In our group, we had excellent coordination. We chatted well. Every member was accountable. They accomplished their work on time. They shared their ideas. They helped other people if need be. Although originally my work was tough since I was the leader, our team was incredibly helpful.

5.5 – Summary of Contribution and Effort

Every individual was a significant element to the success of the system. My role required ensuring there was design consistency. The team was flexible when handling transfer difficulties, validation faults, or design-related concerns.

6 – Conclusion

In conclusion, the GameHaven Online Shop project has proved that ASP.NET MVC can be used to develop a real-world e-commerce site. The site features user registration functionality, user login functionality, user profile maintenance functionality, browse functionality, filtering functionality, and fake checkout functionality.

Implementation of MVC ideas paired with Entity Framework helps develop a well-structured system with suitable focus areas separated. As Lead Developer, duties included system architecture design, database design, admin module development, and security-related features. GitHub allows teamwork to result in simple joining.

As for future development, offering real-world payment options like Stripe or PayPal features, or adding filters or algorithms for cloud-based storage like Azure would go a long way. In summary, not only did our project boost our coding talents but our team-working skills as well as we've managed to construct a safe user-friendly web app that matches professional coding standards.

7 – References

Esposito, D. (2011). *Programming Microsoft ASP.NET MVC*. O'Reilly Media, Inc.

Fowler, M., Rice, D., Foemmel, M., Hieatt, E., Mee, R., Stafford, R. and Wesley, A. (2002). *Introduction Architecture Enterprise Applications Kinds of Enterprise Application Thinking About Performance Patterns*. [online] Available at: https://dl.ebooksworld.ir/motoman/Patterns%20of%20Enterprise%20Application%20Architecture.pdf.

Kenneth C, Laudon and Carol Guercio Traver (2019). *Ecommerce Business Technology Society 17th Edition Standalone Book 17th Edition Laudon download*. [online] Scribd. Available at: https://www.scribd.com/document/891868032/Ecommerce-Business-Technology-Society-17th-Edition-Standalone-Book-17th-Edition-Laudon-download [Accessed 9 Nov. 2025].

Loeliger, J. (2022). *Version Control with Git, 3rd Edition*. [online] O'Reilly Online Learning. Available at: https://www.oreilly.com/library/view/version-control-with/9781492091189/ [Accessed 9 Nov. 2025].

Microsoft (2021). *Overview of Entity Framework Core - EF Core*. [online] learn.microsoft.com. Available at: https://learn.microsoft.com/en-us/ef/core/.

Microsoft (2022). *Intro to ASP.NET MVC*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/getting-started-with-mvc/getting-started-with-mvc-part1.

Microsoft (2024). *Databases - SQL Server*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/sql/relational-databases/databases/databases?view=sql-server-ver17.

OWASP (2017). *Authentication · OWASP Cheat Sheet Series*. [online] Owasp.org. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html.

Ruas, T. and Grosky, W.I. (2020). *Software Engineering: A Practitioner's Approach 9 th Edition*. [online] Research Gate. Available at: https://www.researchgate.net/publication/365946272_Software_Engineering_A_Practitioner.

Stuttard, D. and Pinto, M. (2025). *elhacker.INFO - 404 página no encontrada - Descarga Curso, Manual, Tutorial o Libro not found*. [online] Elhacker.info. Available at: https://elhacker.info/manuales/Cybersecurity%20Books/The%20web%20application%20hackers%20handbook%20discovering%20and%20exploiting%20security%20flaws%20by%20Dafydd%20Stuttard.

8 – Appendix

Appendix 1 – Home Page Wireframe

The design of these wireframes takes into account the necessity for all pages of the GameHaven Online Shop to have a consistent and user-friendly interface that matches the conventional e-commerce site structure and functionality. All pages of this online store adopt a predictable pattern that is simple for visitors to travel through while they purchase online.

Below is a description of all wireframes and interfaces used on each page of this online business.

Figure 14 - Home Page Wireframe

The Home Page is the key entrance point of this system, and this has been built with a simple and intelligible structure in mind. Right at the top of this page, there is a spot on the left for the GameHaven Logo and then comes GameHaven itself. To its right, there is Login, Register, and Basket.

Under the header, there is a navigation bar that lists four primary categories of consoles: Nintendo, PlayStation, Xbox, and Computer, and below this is the Search Bar that enables one to search for products based on keywords, and then there is Sort By, which enables one to sort products based on price (Low to High/High to Low and A-Z/Z-A).

The central half of this webpage is dominated by a cycling advertising banner promoting current deals, followed by Best Sellers, highlighting popular games accessible on this platform. All this is done to allow users to simply see promotional items while acquainting them with this store's promotion material as well.

Appendix 2 – Nintendo Main Page Wireframe

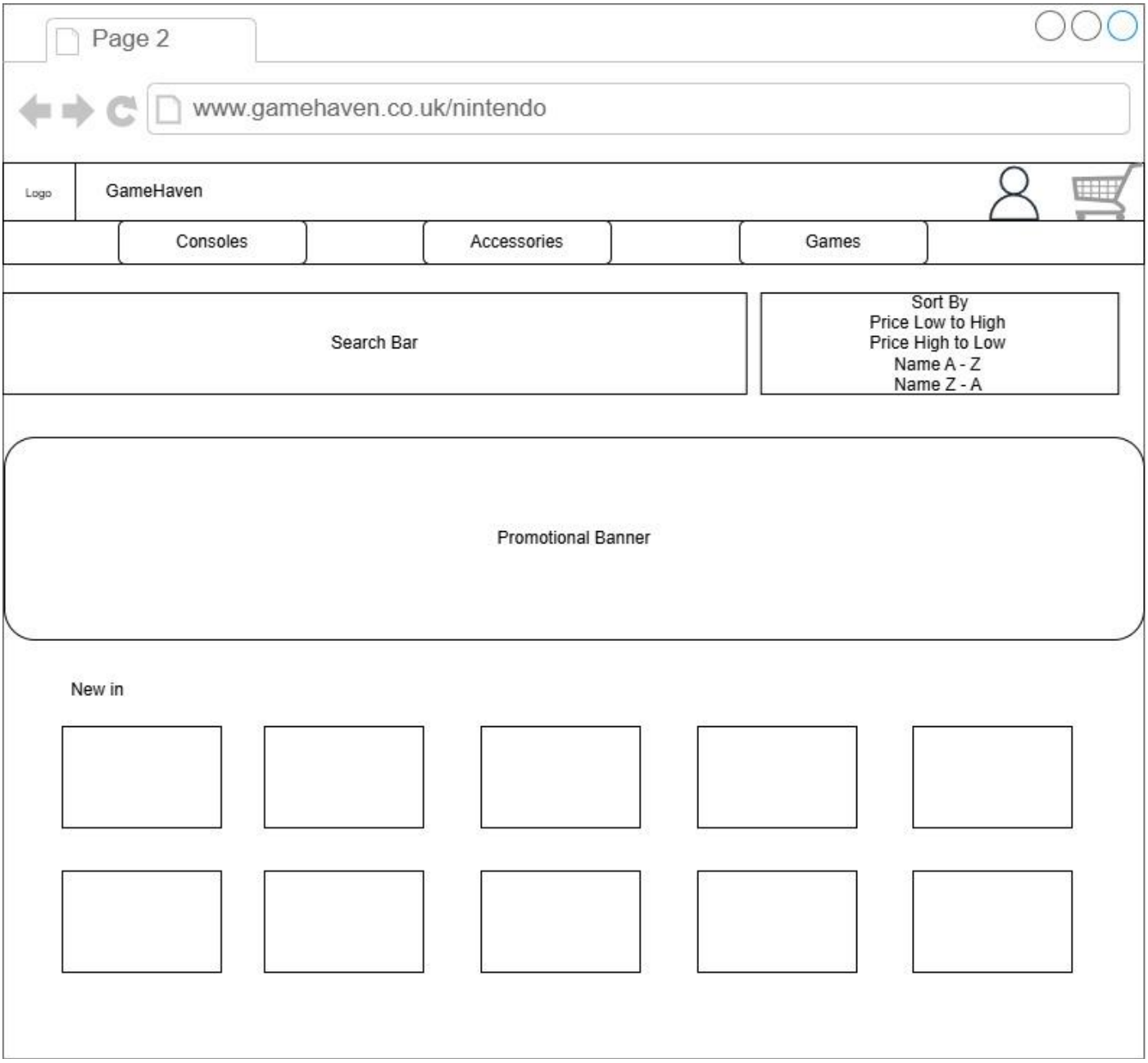


Figure 15 - Nintendo Main Page Wireframe

The Nintendo Main Page is a copy of the homepage design, particularly dedicated to Nintendo merchandise. Even the header and menu bar stay untouched, offering a consistent user experience. Following the header, clients get easy access to three category buttons, namely Consoles, Accessories, and Games, for restricting their search. There is again a Search Bar and Sort By option below the navigation bar that provides flexible searching and sorting of commodities.

There is a Promotional Banner below, promoting any current Nintendo discounts that are available. Secondly, there is a New In section that reveals any new goods that have just been introduced by Nintendo.

Appendix 3 – Nintendo Consoles Page Wireframe

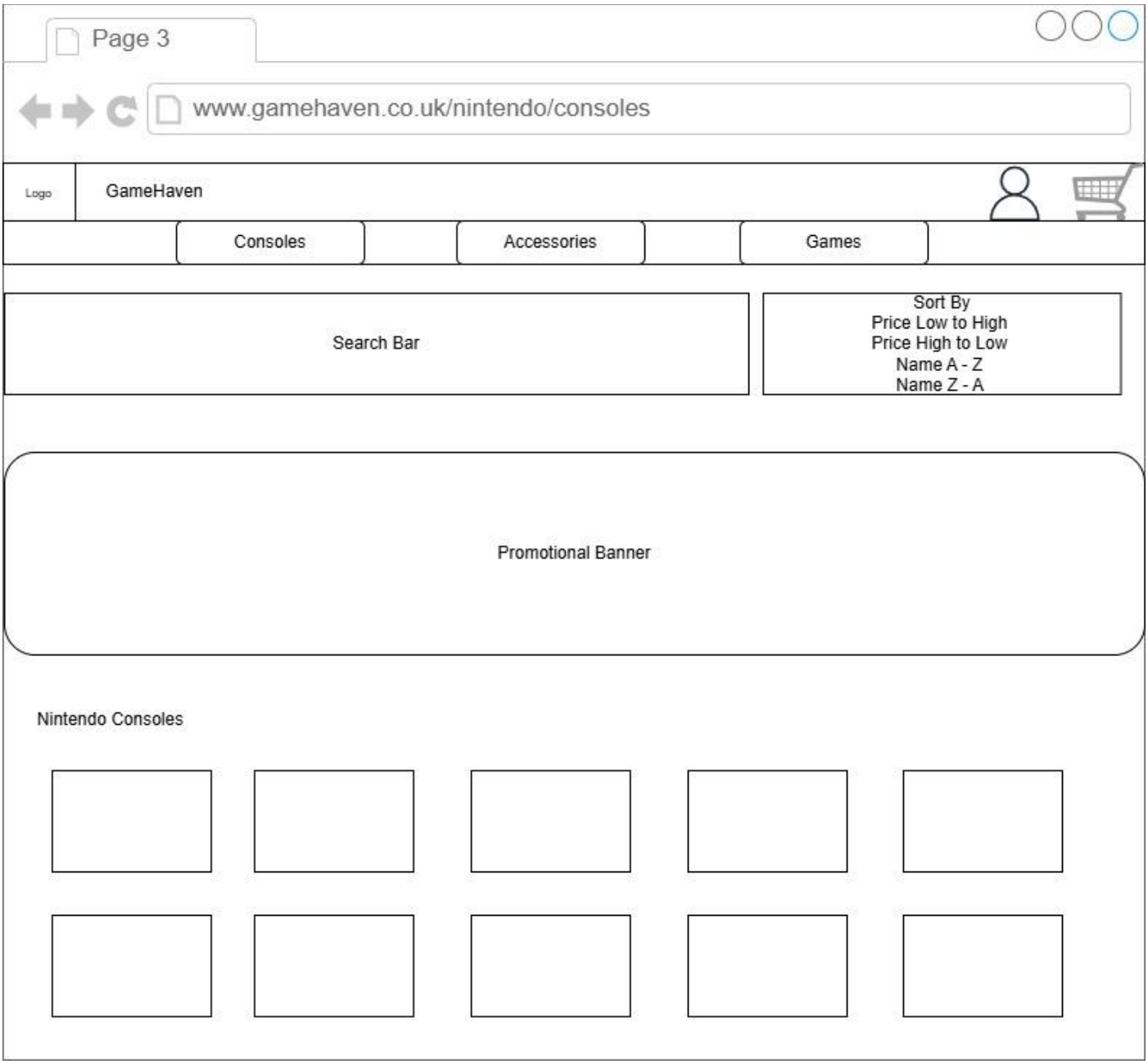


Figure 16 - Nintendo Consoles Page Wireframe

There is continuity in the format of this page, as demonstrated by it having the same header and navigation as the previous prior pages. There is a Search Bar and Sort By tool, and then a Promotional Banner, followed by the section on Nintendo Consoles that lists all available consoles, along with names and prices, and contains photos as well as an Add to Basket option for each one.

Appendix 4 – Nintendo Accessories Page Wireframe

Figure 17 - Nintendo Accessories Page Wireframe

The Nintendo Accessories Page uses precisely the same layout structure as that of the Consoles Page. The Search Bar, Sort By, and Promotion Banner remain unaltered, however the content section is utilised for Nintendo Accessories such as controllers, cases, and headphones. Every product display presents its pricing and photographs well, supporting an educated purchase choice by the consumer.

Appendix 5 – Nintendo Games Page Wireframe

Figure 18 - Nintendo Games Page Wireframe

Nintendo Games Page keeps the same design style that has been utilised before for consistent branding. It contains a Search Bar, Sort By, and Promotion Banner, followed by Nintendo Games, showcasing a list of available games, each of them having a picture, name, and price, and may be put immediately to the basket.

Other Console Pages (Xbox, PlayStation, and Computer) Wireframes

The design chosen by the Xbox, PlayStation, and Computer sites is common, exactly like that of the other Nintendo pages. Every category for each of these consoles has its master site, as well as secondary pages devoted to Consoles, Accessories, and Games, all of which have the same user interface design.

Appendix 6 – Account Login Page Wireframe

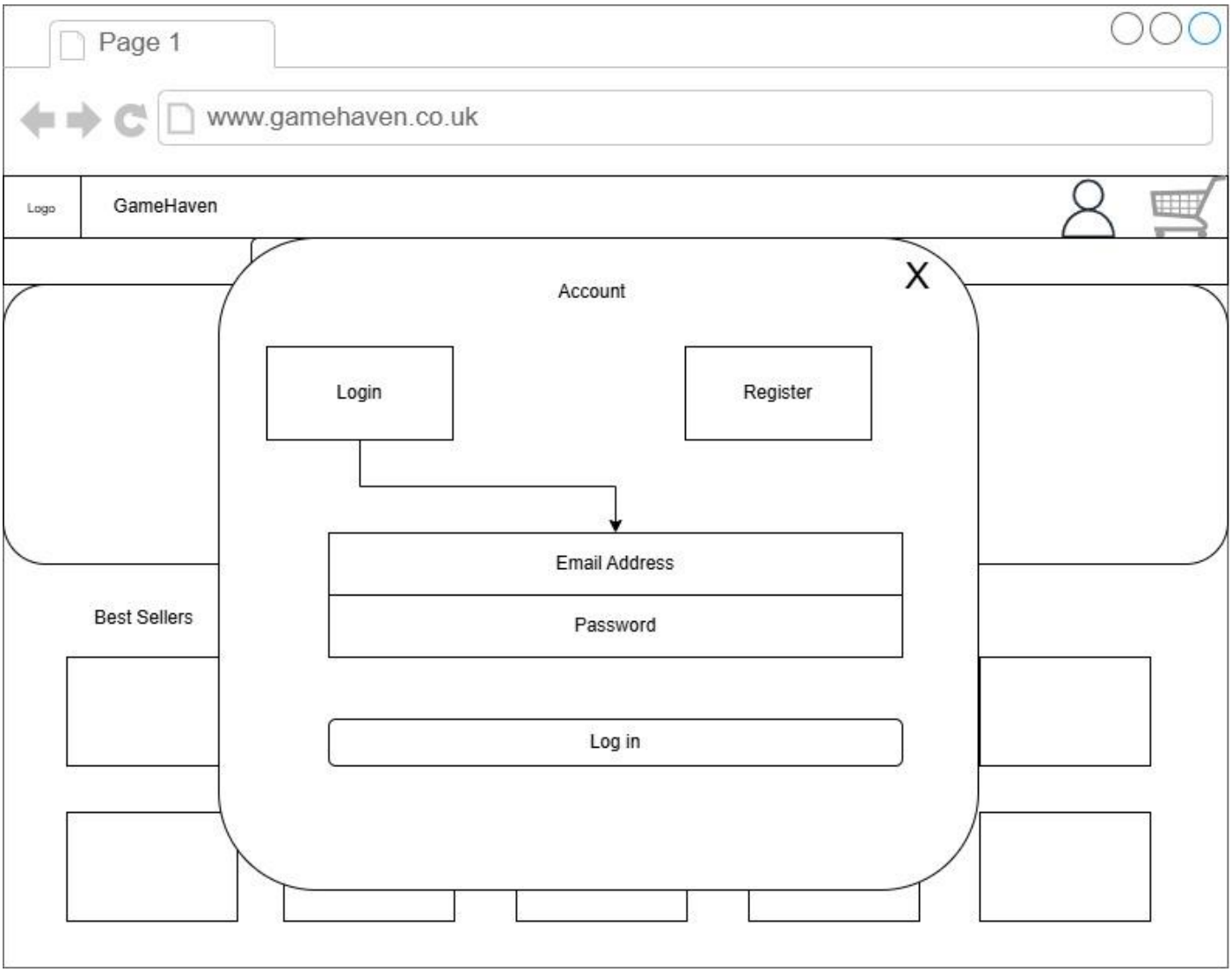


Figure 19 - Account Login Page Wireframe

The Account Login Page is made user-friendly and accessible. When a user clicks on the Login button, they are needed to fill in information of their Email and Password. There is validation on this form that verifies whether all information have really been submitted, and any user supplying wrong login credentials produces a matching alert message.

Appendix 7 – Account Registration Page Wireframe

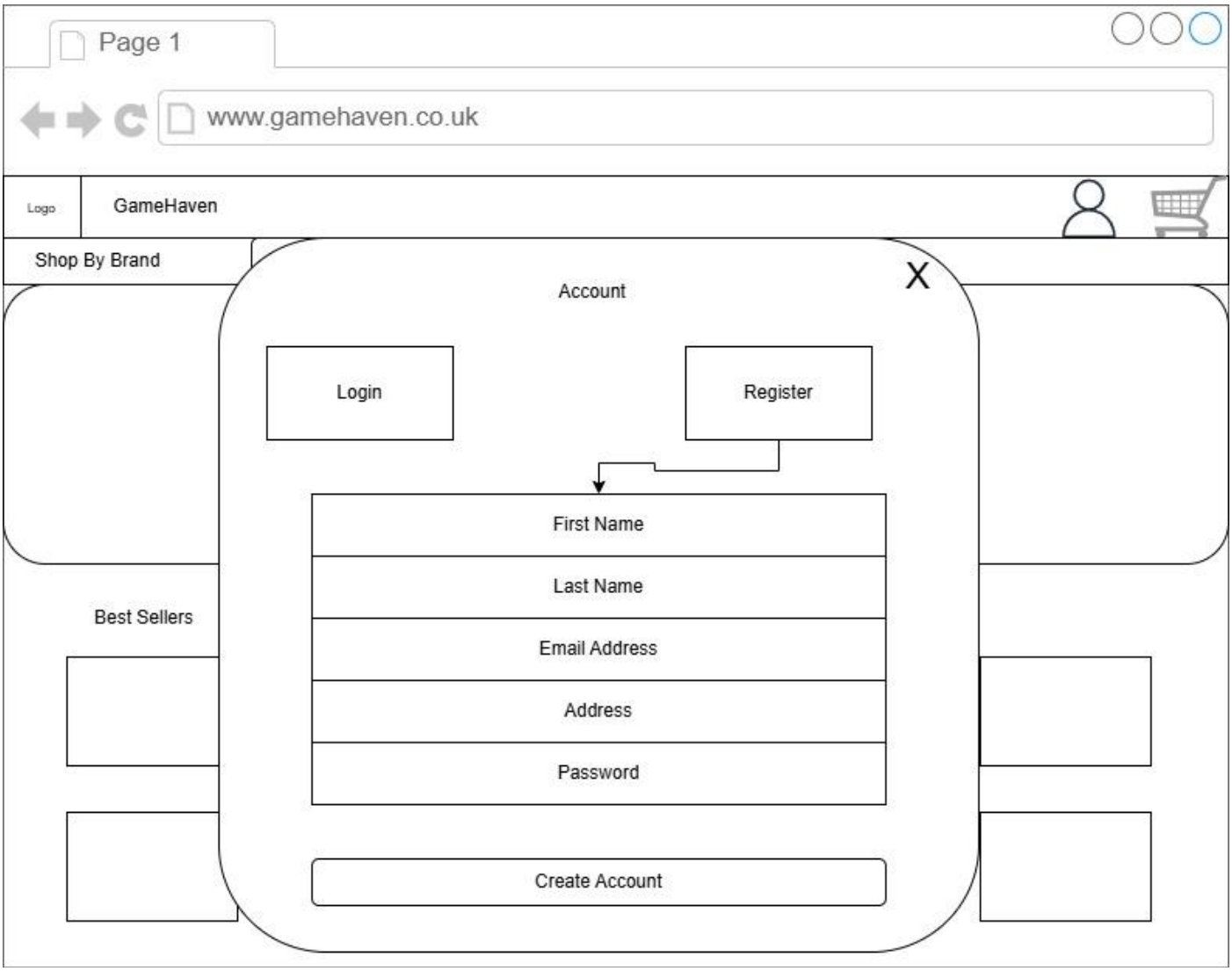


Figure 20 - Account Registration Page Wireframe

For the Account Registration Page, new users may successfully register themselves into the system by giving their first name, last name, email, and password. All this information is subsequently stored into the database using Entity Framework as soon as it is submitted. All this is made feasible by a clear form design supplied on this specific page.

Appendix 8 – Customer Account Page Wireframe

Figure 21 - Customer Account Page Wireframe

Once the user has successfully signed into their system, the users are brought to their Customer Account Page, through which they may access and modify their personal data and prior purchases. The user's name, email, and address are presented on the left side of this page, while their prior orders, dates, things bought, and associated total amounts are displayed on the other side.

Appendix 9 – Admin Dashboard Page Wireframe

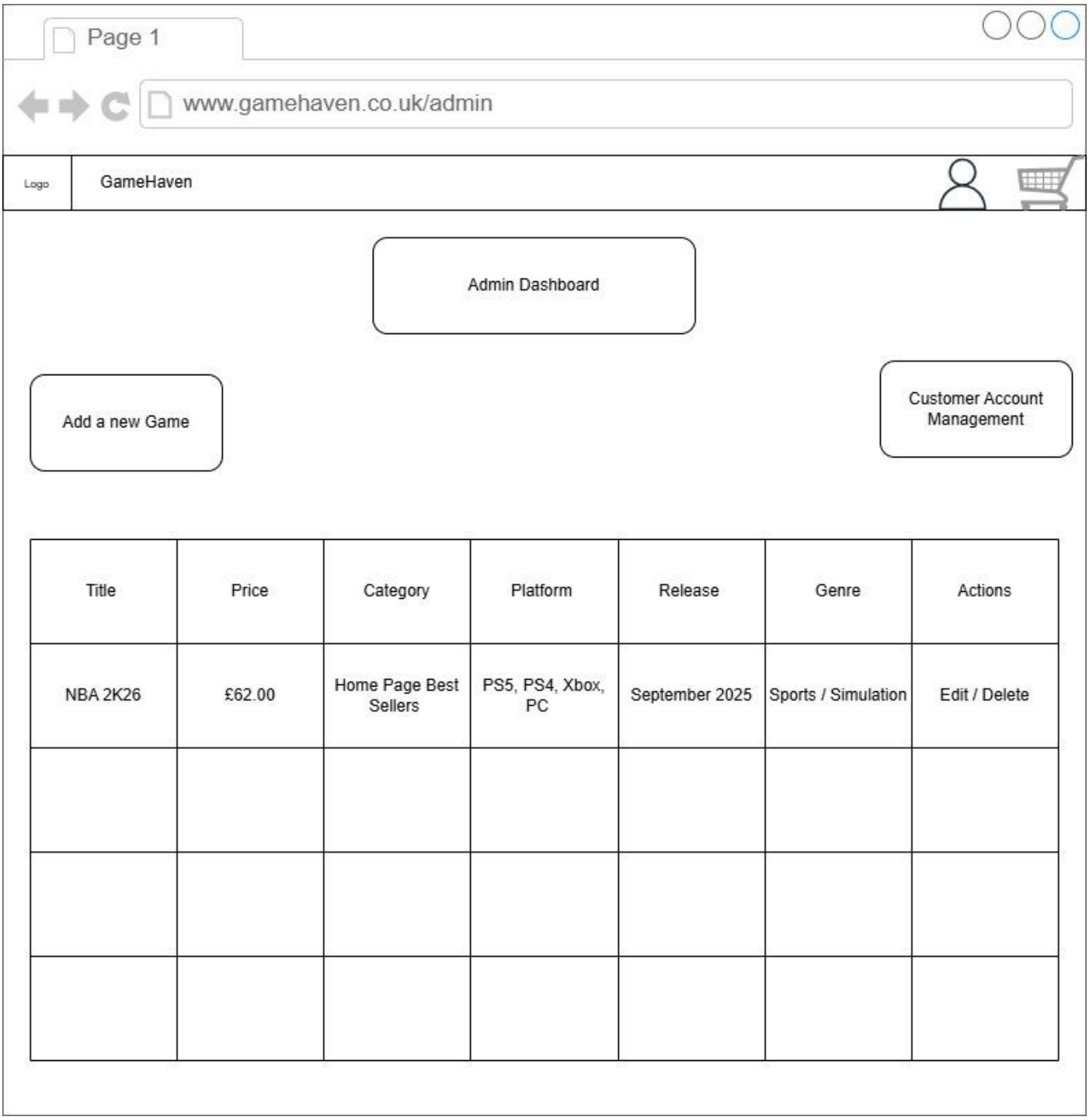


Figure 22 - Admin Dashboard Wireframe

The Admin Dashboard allows administrative users to have comprehensive capabilities of system admin activities. They may examine all listed items, create new games, as well as client accounts and orders. All this is accomplished via an optimised interface that constantly focusses emphasis on efficiency and simple navigation of its functions.

Appendix 10 – Basket Page Wireframe

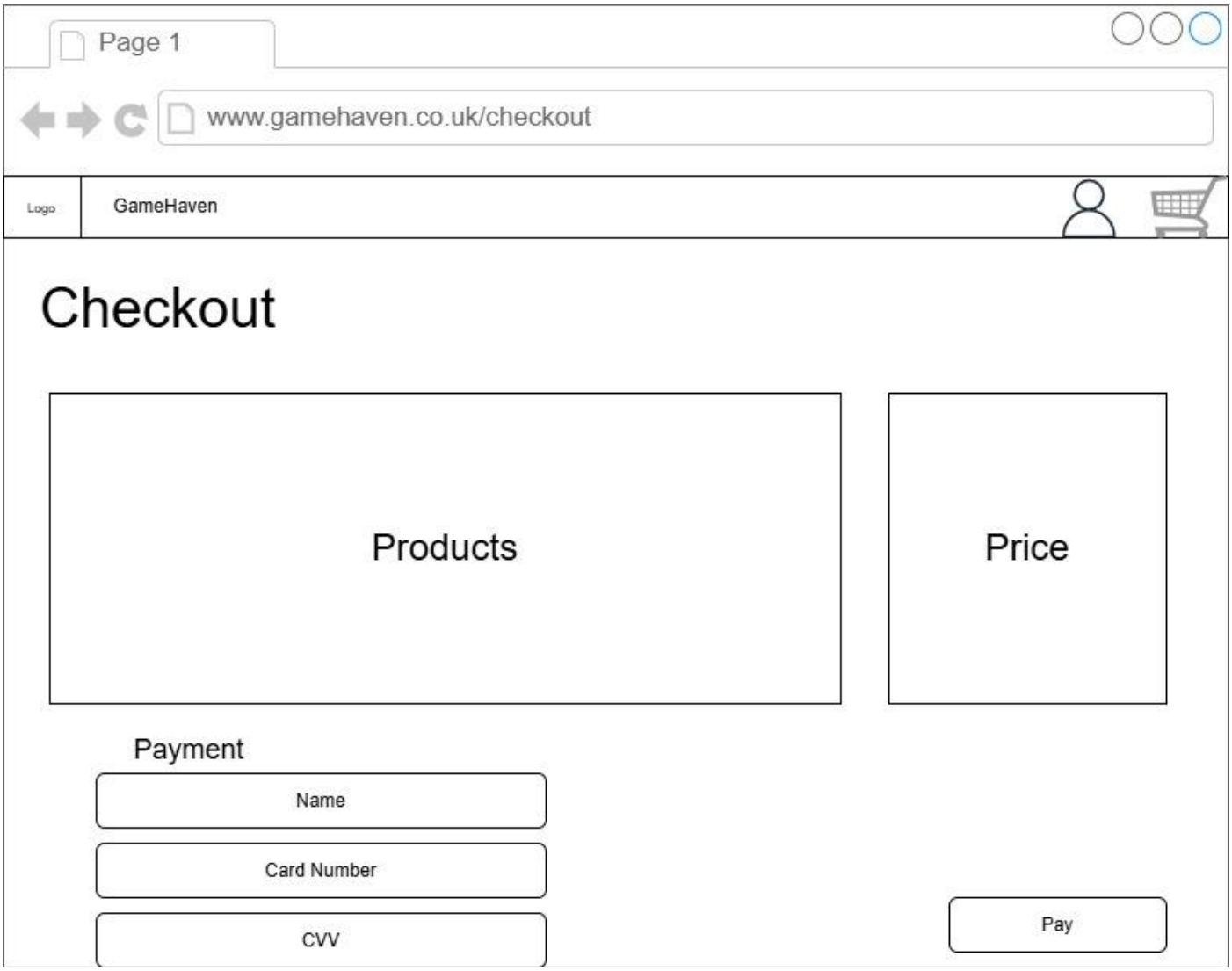


Figure 23 - Basket Page Wireframe

The Basket Page lists all the goods that have been chosen by the user. Every entry on this page gives the name of each item, its quantity, price, and finally its total worth. At the conclusion of this page, there are two buttons that operate as actions, namely Continue Shopping and Checkout buttons that transfer customers to Home Page and the Payment page, respectively, as they go towards paying for their purchases.

Appendix 11 – Checkout Page Wireframe

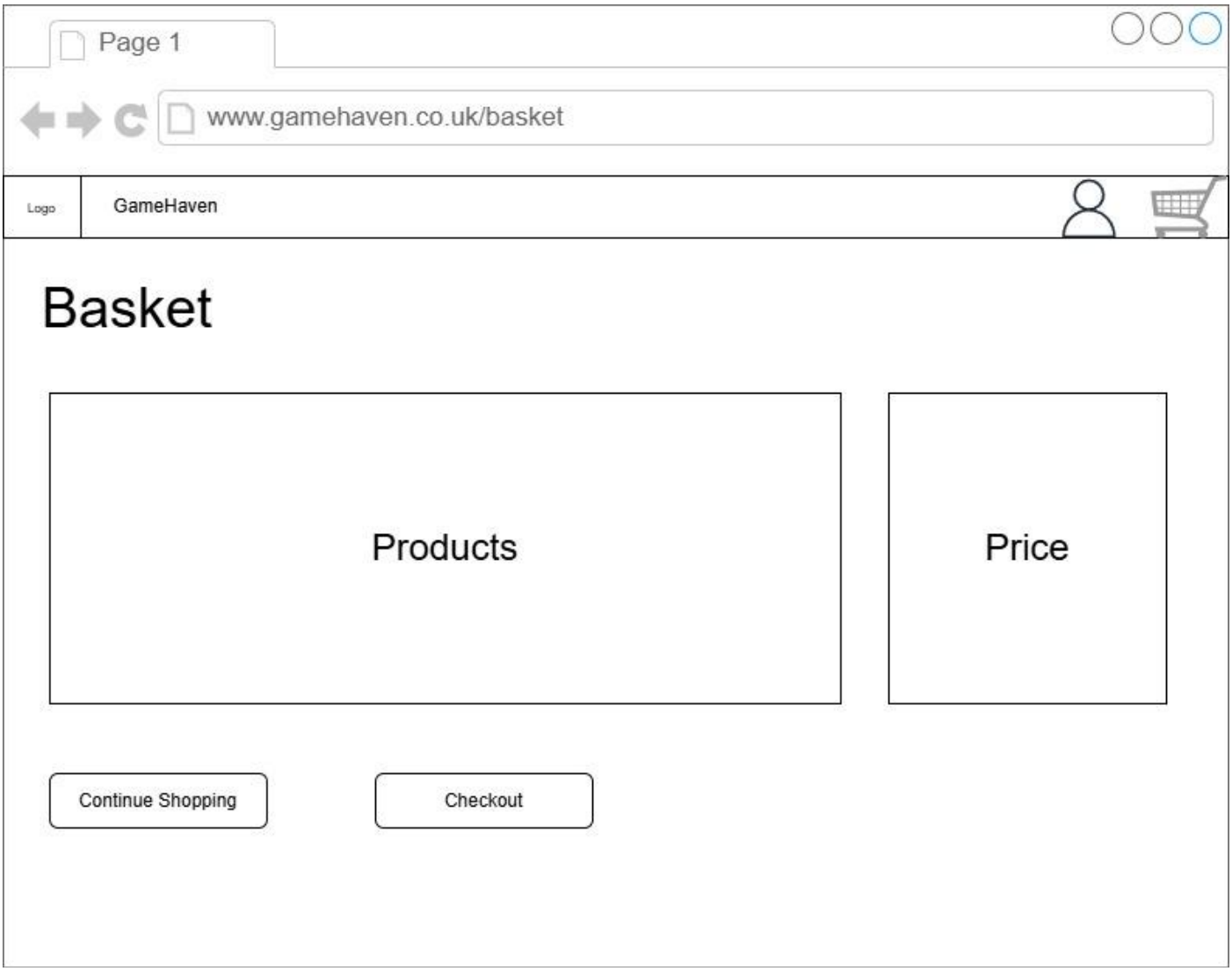


Figure 24 - Checkout Page Wireframe

The Checkout Page is the last phase of purchase as a user may browse things he has added, verify pricing, and continue to enter payment information such as credit card number, expiry date, and CVV number. The site is created linearly to prevent any misunderstanding on the part of the user. Even though a fake payment gateway system has been employed, this architecture is suitable with payment integration solutions as well.

In summary, every wireframe was made uniform and featured a same header, navigation bar, and layout hierarchy that helped create a unified browsing experience on all pages of each site. By doing this, whether one is browsing items or one's own account, all UI elements become familiar and consistent as well. Therefore, by retaining this continuity, this design promotes usability and helps comply to fundamental criteria of accessibility and design that are followed on current websites nowadays.

Appendix 12 – Final System

The next section illustrates the final version of the GameHaven Online Shop system that was actually placed into operation. Each component represents how the finished interfaces of the system appear. The system has progressed from wireframes to an actual functional software. The final system maintains the same structure, process, and interface as in the original design concept. However, it is much more usable, accessible, and functional across all devices. The most significant aims in design, including usable interfaces, a strong brand, and a secure, efficient, and reliable system architecture, have been achieved.

Appendix 12.1 – Main Home Page

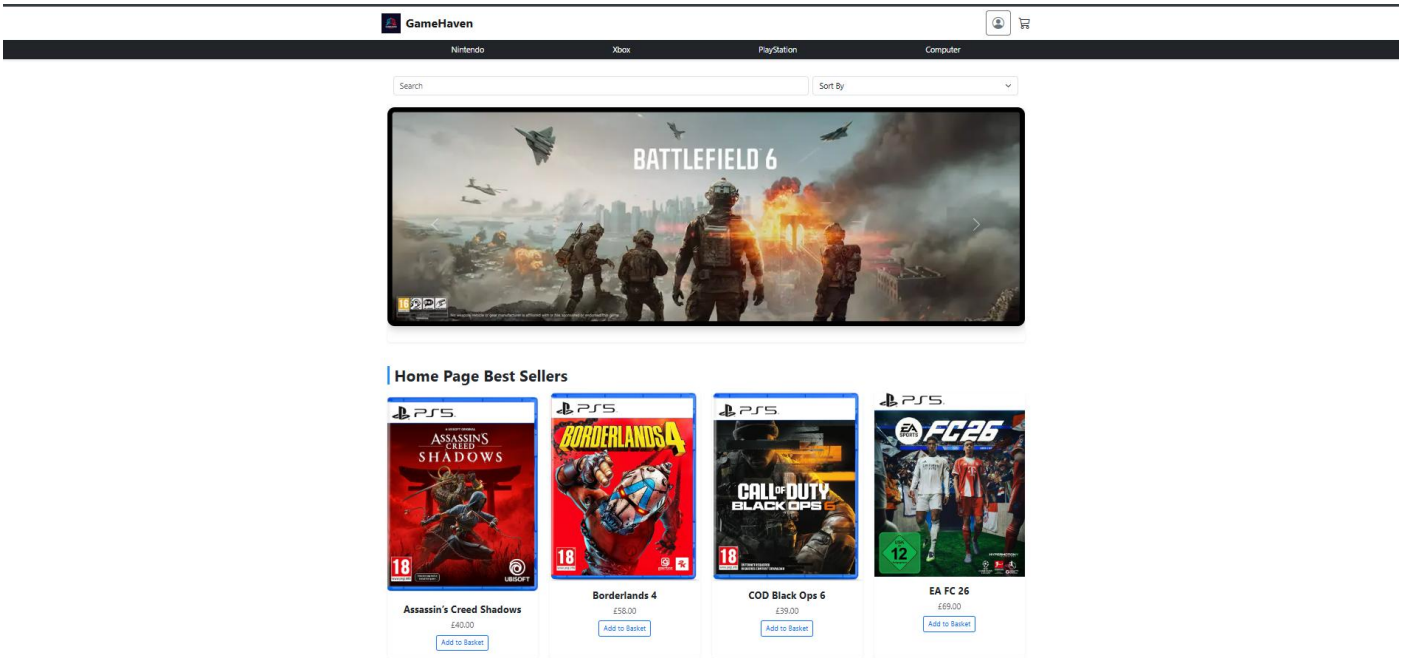


Figure 25 - Final System – Main Home Page

The website major page is the Home Page, which is designed to be clean and straightforward to use. The GameHaven logo appears in the upper left corner of the top banner. The Login, Register, and Basket buttons for important navigation appear in the top right corner of this e-commerce website for games. Under the top banner, there is a horizontal navigation bar which shows all the categories of products we have inside our e-commence game website. The categories which we have on our website is as follows, we have Nintendo, PlayStation, Xbox and Computer. The primary area of content displays advertising banners that signal discounts that are presently going on, as well as a Best Sellers section that shows popular goods. This is aimed at giving an attractive introductory page to visitors.

Appendix 12.2 – Nintendo Main Home Page

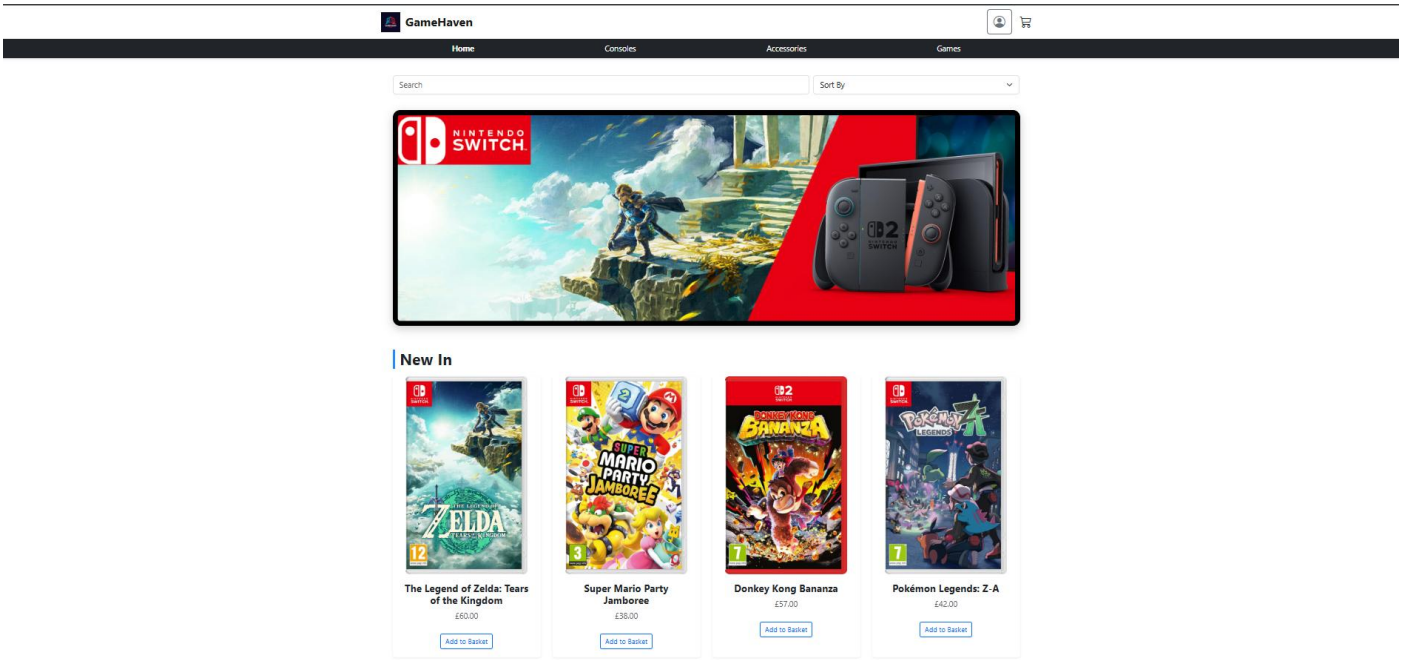


Figure 26 - Final System – Nintendo Main Home Page

The Nintendo Page is designed in the same method as the Home Page, keeping consistency across the platform. The Nintendo Page is focused to Nintendo things alone, delivering easy-to-click links for Consoles, Accessories, and Games, respectively. A marketing banner also reveals current Nintendo specials, and a New In section denotes brand-new products that have been launched for sale.

Appendix 12.3 – Xbox Consoles Page

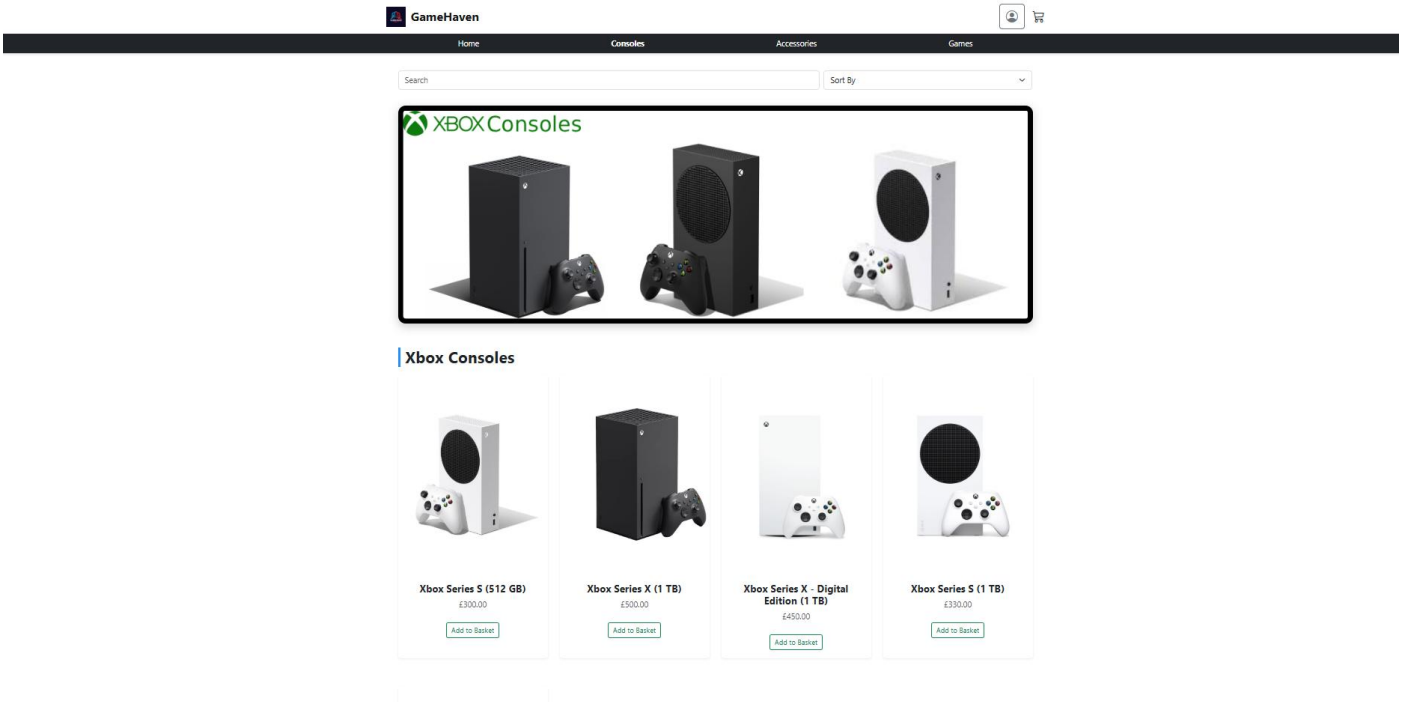


Figure 27 - Final System – Xbox Consoles Page

The Xbox Consoles Page preserves the same layout for all pages of the website, including the header, navigation, search facility, and choices for ordering and filtering things from high to low, low to high, and so on. The principal area of this site is split into various consoles of Xbox things, which contain a picture of a product, name, price, and a facility to add to the basket.

Appendix 12.4 – PlayStation Accessories Page

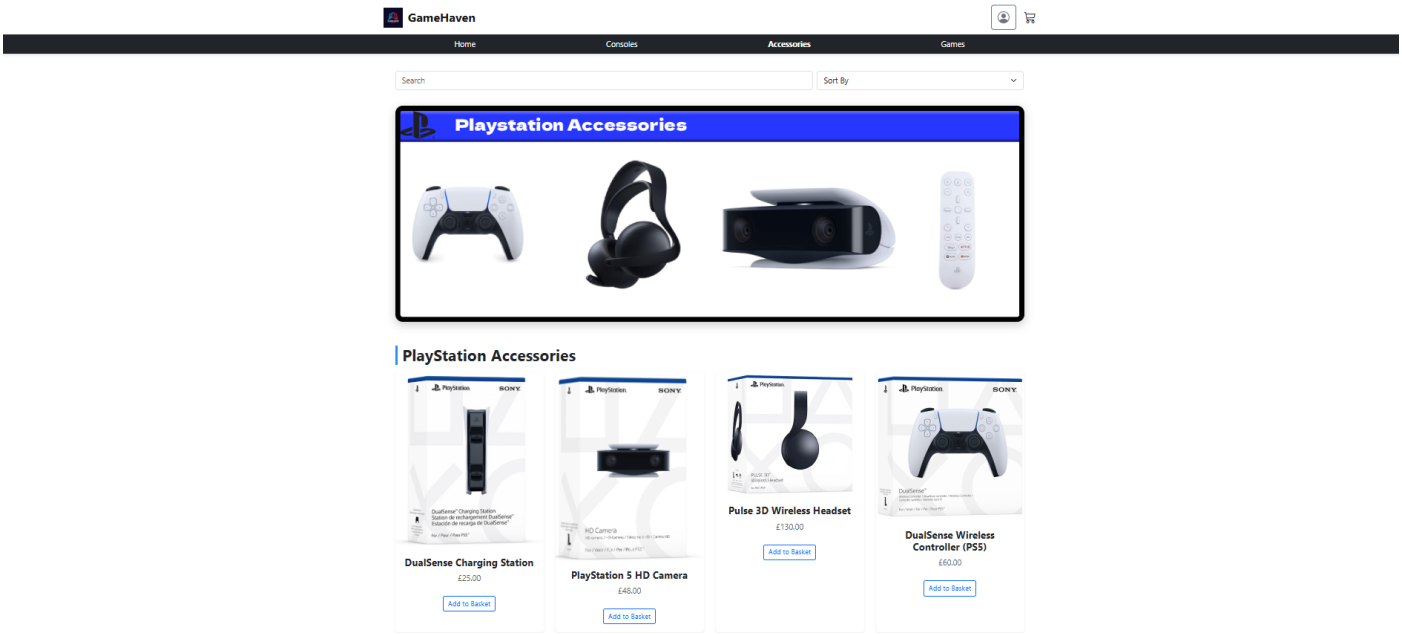


Figure 28 - Final System – PlayStation Accessories Page

The PlayStation Accessories Page is built in a fashion analogous to other sections on the website. The top half of this site retains aspects such as the navigation bar, search box, as well as information about promotions, to offer a uniform experience for all end-users. The part below gives information on accessories accessible for playing stations, including goods such as controllers, headsets, as well as carry cases, which indicate unique pictures, price, as well as purchasing outlets for all products.

Appendix 12.5 – Computer Games Page

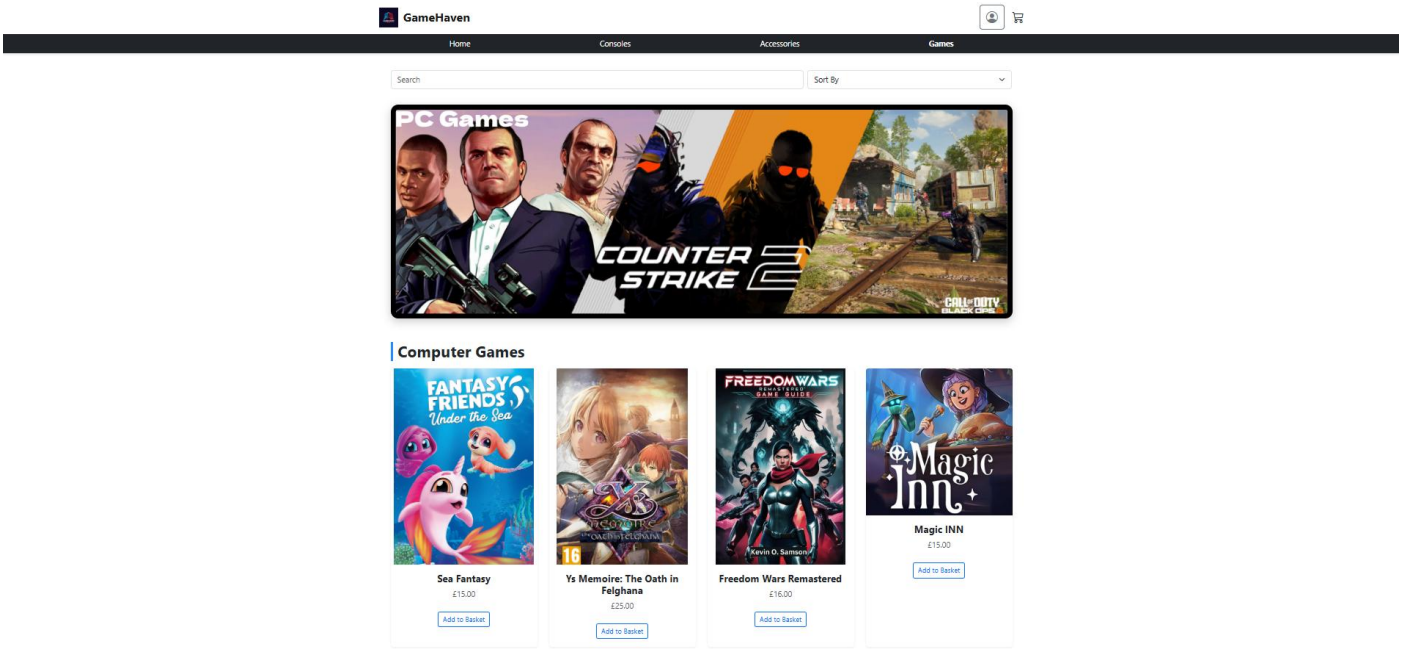


Figure 29 - Final System – Computer Games Page

The Computer Games Page is in agreement with a comparable visual brand maintained across the internet platform. The website also facilitates searching, filtering, as well as viewing available discounts, employing the promotion banner situated at the top of the page. The principal web page comprises computer games, accompanied by cover photos, titles, as well as price, enabling convenient addition to user's purchase carts.

Appendix 12.6 – Basket Page

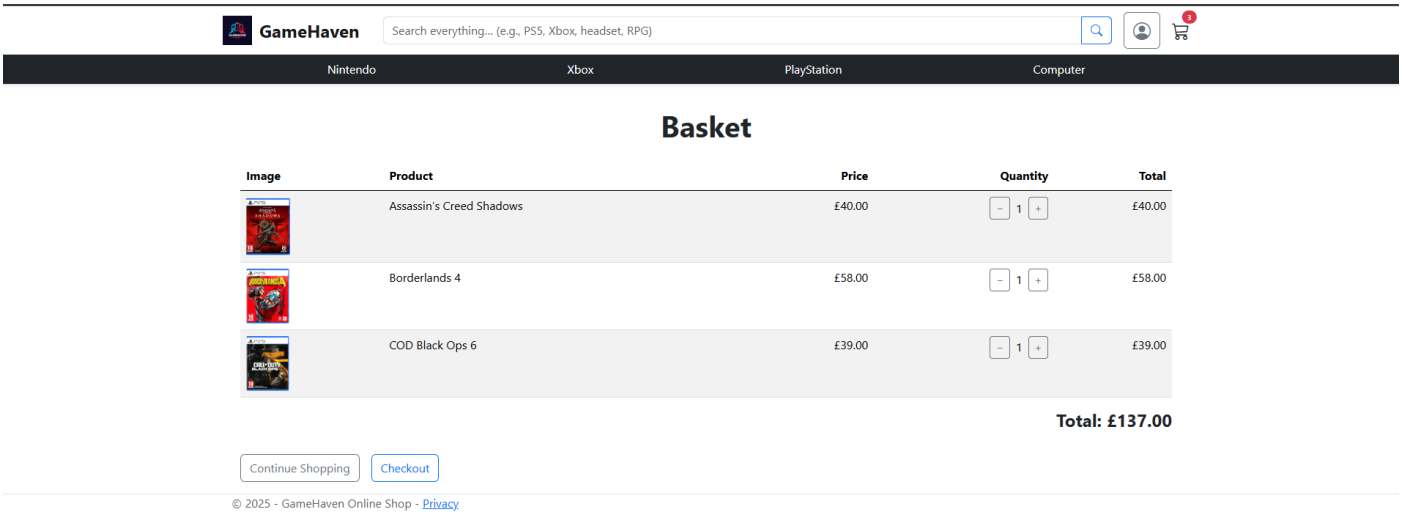


Figure 30 - Final System – Basket Page

The Basket Page presents a detailed summary of all things added by the user to his/her basket. For every product, buyers find details such as the name, quantity, unit price, as well as total price, which display when consumers add things to their baskets. Finally, at the finish of the page, visitors find two comprehensive buttons to either continue shopping or proceed to check out, consequently improving client convenience while they buy.

Appendix 12.7 – Customer Dashboard

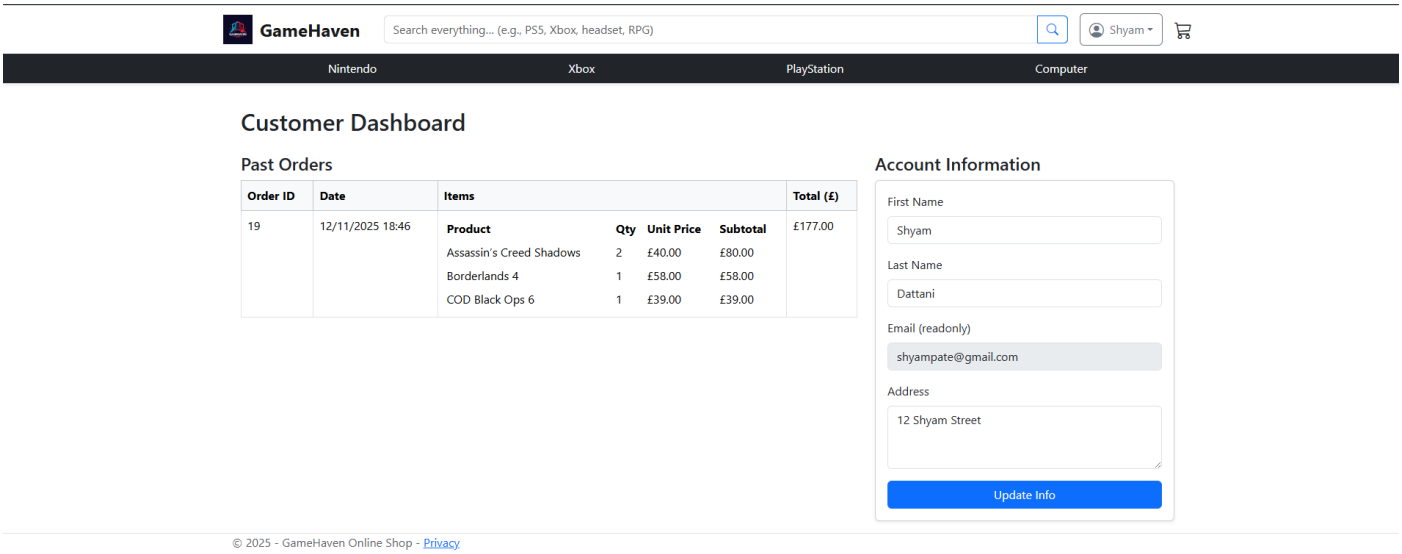
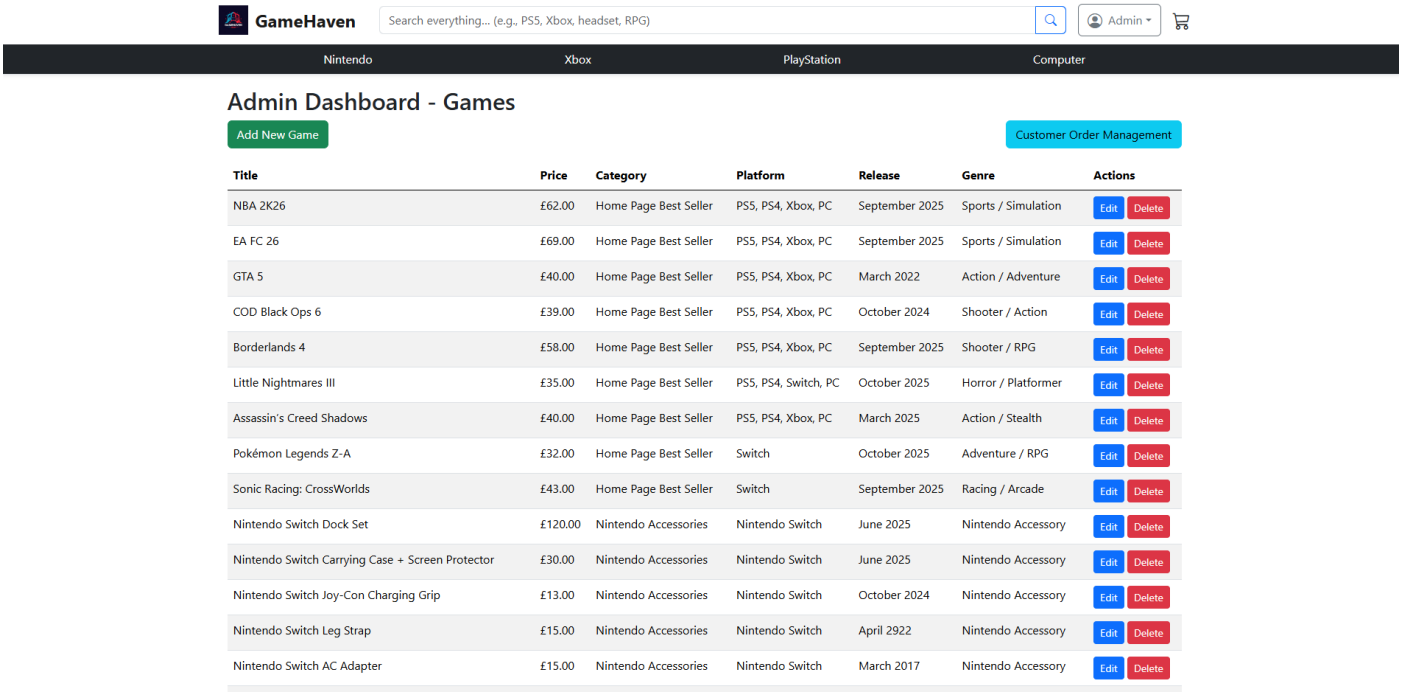


Figure 31 - Final System – Customer Dashboard

After login, customers are redirected to a page referred to as the Customer Dashboard, where they have an opportunity to check and correct their details as well as those of any orders they may have placed. The page

gives information about clients, including name, email, and addresses, in connection to orders they made in the past, as evidenced by dates, items, and total amounts.

Appendix 12.8 – Admin Dashboard



Title	Price	Category	Platform	Release	Genre	Actions
NBA 2K26	£62.00	Home Page Best Seller	PS5, PS4, Xbox, PC	September 2025	Sports / Simulation	Edit Delete
EA FC 26	£69.00	Home Page Best Seller	PS5, PS4, Xbox, PC	September 2025	Sports / Simulation	Edit Delete
GTA 5	£40.00	Home Page Best Seller	PS5, PS4, Xbox, PC	March 2022	Action / Adventure	Edit Delete
COD Black Ops 6	£39.00	Home Page Best Seller	PS5, PS4, Xbox, PC	October 2024	Shooter / Action	Edit Delete
Borderlands 4	£58.00	Home Page Best Seller	PS5, PS4, Xbox, PC	September 2025	Shooter / RPG	Edit Delete
Little Nightmares III	£35.00	Home Page Best Seller	PS5, PS4, Switch, PC	October 2025	Horror / Platformer	Edit Delete
Assassin's Creed Shadows	£40.00	Home Page Best Seller	PS5, PS4, Xbox, PC	March 2025	Action / Stealth	Edit Delete
Pokémon Legends Z-A	£32.00	Home Page Best Seller	Switch	October 2025	Adventure / RPG	Edit Delete
Sonic Racing: CrossWorlds	£43.00	Home Page Best Seller	Switch	September 2025	Racing / Arcade	Edit Delete
Nintendo Switch Dock Set	£120.00	Nintendo Accessories	Nintendo Switch	June 2025	Nintendo Accessory	Edit Delete
Nintendo Switch Carrying Case + Screen Protector	£30.00	Nintendo Accessories	Nintendo Switch	June 2025	Nintendo Accessory	Edit Delete
Nintendo Switch Joy-Con Charging Grip	£13.00	Nintendo Accessories	Nintendo Switch	October 2024	Nintendo Accessory	Edit Delete
Nintendo Switch Leg Strap	£15.00	Nintendo Accessories	Nintendo Switch	April 2922	Nintendo Accessory	Edit Delete
Nintendo Switch AC Adapter	£15.00	Nintendo Accessories	Nintendo Switch	March 2017	Nintendo Accessory	Edit Delete

Figure 32 - Final System – Admin Dashboard

The Admin Dashboard offers a well-structured design to facilitate administration of users, commodities, as well as orders. Administrators have abilities of upgrading games, analysing registered customers, as well as transaction monitoring in an appropriate approach. The system is structured in a manner that facilitates easy movement between the different administrative functions.

Appendix 12.9 – Admin Customer Order Management

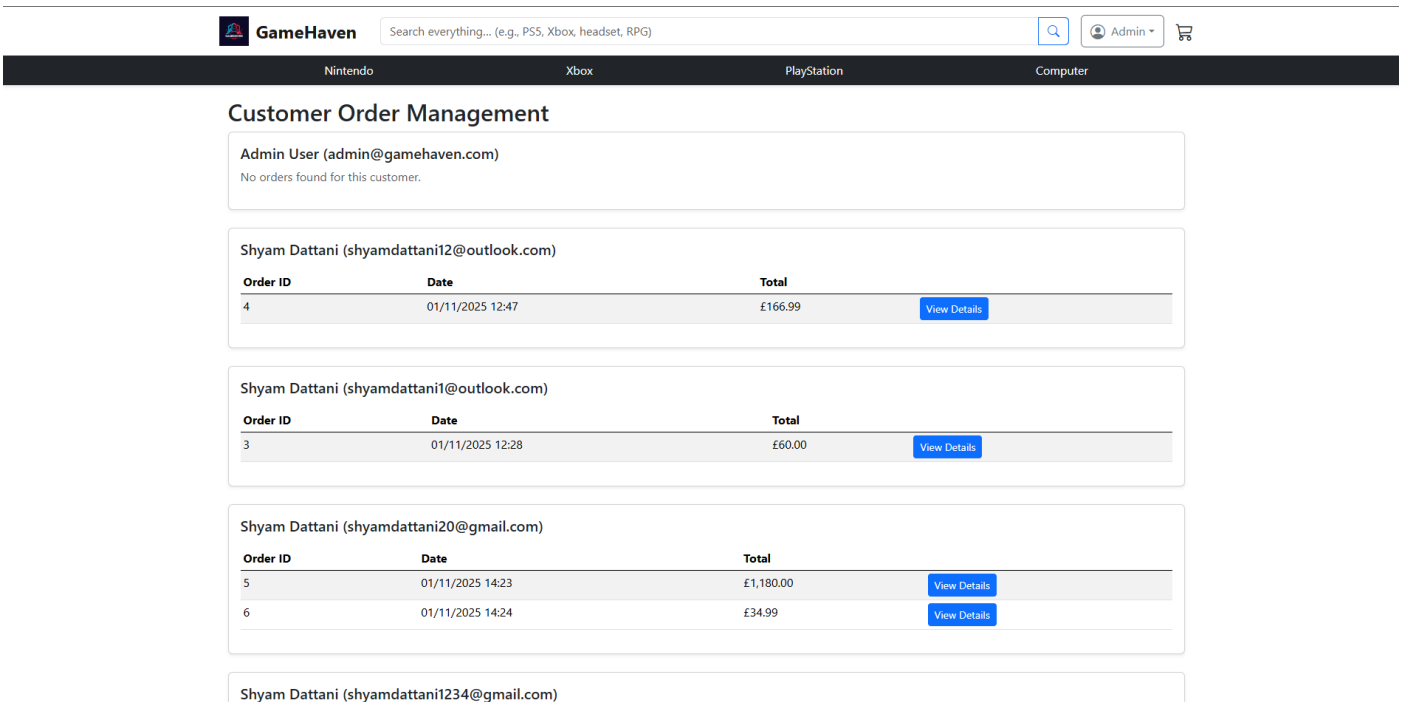


Figure 33 - Final System – Admin Customer Order Management

The selection of the customer Order Management button from the Admin Dashboard results in a page presenting an orderly table of all client orders done by them. Every entry in this table consists Order ID, Date, Total Amount, Customer Name, and an Email Address. Moreover, administrators now have an option to View Details of an order by clicking on it, which delivers an informative site for more in-depth examination of purchases made by customers.

Appendix 12.10 – Admin Customer Order Management View Details

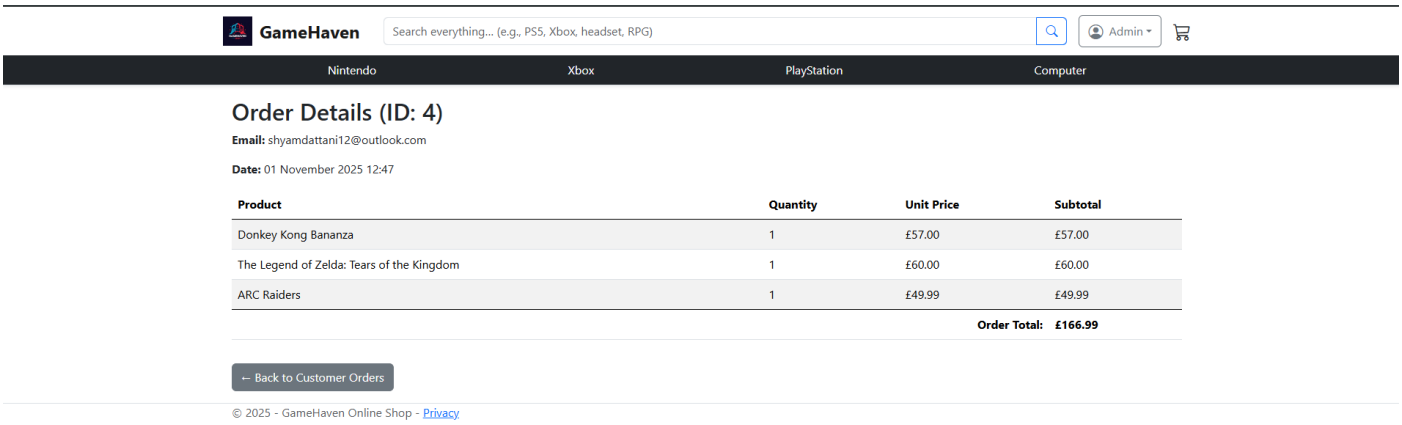


Figure 34 - Final System – Admin Customer Order Management View Details

When an administrator chooses View Details, a complete examination of the order is given by the system. This comprises Order ID, Customer's Email, Order Date, as well as a table providing data of all items ordered, together with their costs, inclusive of a total order amount. A link to Back to Customer Orders provides a transition to the previous page, ensuring efficiency in details examination by administrators.

In summary, the final implementation of GameHaven Online Shop is a successful transition of concept wireframes to a functional web-based system. Each page of this system maintains original design concepts in terms of being consistent, responsive, and functional, in addition to including real data, login authentication, and easy movement from page to page, which enables a perfect blend of design concepts and functionality to provide a professional look as well as a functional experience for customers as well as administrators.